

Codebase Walkthrough

Purpose

This document explains the repository through real end-to-end product flows.

Instead of listing APIs separately, each flow shows:

- where the user starts in the frontend
- which frontend function runs
- which API is called at that exact step
- what payload is sent
- which backend router receives it
- which backend functions/files continue the work
- what response comes back
- how the frontend uses that response

This makes the codebase easier to read in execution order rather than in file order.

Related references:

- [BACKEND_ARCHITECTURE_REPORT.md](#)
- [FRONTEND_FULL_WALKTHROUGH.md](#)

Project Summary

This repository is an AI-assisted analytics platform built around two related problems that most organizations have:

- business data is often trapped in Excel and CSV files, which are difficult to understand quickly without manual analysis
- operational databases are often noisy, redundant, and not directly ready for business users to query safely

The project solves those problems in two ways:

- file-centric analytics: users can upload Excel/CSV files, preview them, chat with them in natural language, generate dashboards, compare datasets, and organize them into projects and boards
- workspace-centric analytics: users can connect source databases, extract data, clean and transform it, load curated outputs into isolated workspace schemas, define business semantics, then chat/report/dashboard over that curated workspace

At a business level, the system acts like an internal analytics assistant plus dashboard studio:

- it reduces dependency on external third-party tools for first-pass understanding of internal data
- it lets teams move from raw tables or spreadsheets to insights faster
- it preserves organizational context through semantic definitions, reusable dashboards, saved flows, and shareable outputs
- it keeps the organization's data workflows inside its own governed application boundary

Tech Stack

Frontend

- Vue 3
- TypeScript
- Vite
- Vue Router
- Pinia
- Axios and `fetch`
- GridStack for dashboard layout
- ECharts and chart components for visualizations
- Tailwind CSS
- shadcn-vue style UI components

Backend

- Python
- FastAPI
- Uvicorn
- Pydantic models for request/response validation
- `python-multipart` for uploads
- `httpx` and `requests`

Data and ETL

- `pandas`

- `openpyxl`
- `pyarrow`
- SQLAlchemy
- PostgreSQL
- MySQL
- MongoDB

AI and semantic layer

- LangChain
- `langchain-openai`
- `langchain-ollama`
- `langchain-community`
- `langchain-experimental`
- pgvector
- custom embeddings integration

Auth, cache, and observability

- JWT
- `bcrypt`
- `passlib`
- Redis semantic cache
- Langfuse tracing and session observability

Repository Layout

At the top level, the workspace is split into frontend, backend, and documentation:

```
pandas triagent system/  
├─ docs/  
├─ excel-ai-client/  
└─ excel-ai-server-api/
```

`excel-ai-client/`

Vue frontend application.

Important areas:

- `src/pages/` Route-level product screens like upload, documents, ETL, workspace chat, workspace report, workspace dashboard, and `SmartDashboard.vue`.
- `src/services/` API integration layer, mainly `excelApi.ts` and `workspaceApi.ts`.
- `src/store/` Pinia stores for auth, documents, boards, and shared state.
- `src/components/` Reusable UI, dashboard widgets, layout components, and chart views.
- `src/router/` Route definitions and auth guard behavior.
- `src/utils/` Helpers like file-tree construction and Langfuse auth bridging.

`excel-ai-server-api/`

FastAPI backend application and ETL/AI engine.

Important root-level files:

- `main.py` FastAPI entrypoint and router registration.
- `.env` Runtime configuration.
- `requirements.txt` Python dependencies.
- `users.json` Local user/auth storage used by the current auth implementation.
- `setup_pg_role.py` PostgreSQL setup helper.
- `docker-compose-langfuse.yml` Optional Langfuse stack.
- `PROJECT_STRUCTURE.md` Backend-oriented structure notes.

Backend Folder Structure

Expanded Tree View

```
excel-ai-server-api/  
├─ .env  
├─ .gitignore  
├─ QUICK_START_GUIDE.md  
├─ README.md  
└─ SETUP_INSTRUCTIONS.md
```

```
| database.db
| docker-compose-langfuse.yml
| main.py
| requirements.txt
| setup_pg_role.py
| start-langfuse.bat
| users.json
| exports/
| venv/
| app/
|   | __init__.py
|   | config.py
|   | dependencies.py
|   | core/
|   |   | __init__.py
|   |   | agent.py
|   |   | auth.py
|   |   | cache.py
|   |   | callbacks.py
|   |   | database.py
|   |   | embeddings.py
|   |   | file_ops.py
|   |   | formatters.py
|   |   | ingestion.py
|   |   | llm.py
|   |   | routing.py
|   |   | schema_utils.py
|   |   | agents/
|   |   |   | __init__.py
|   |   |   | profiler.py
|   |   |   | workspace_sql.py
|   |   | dashboard/
|   |   |   | __init__.py
|   |   |   | kpi_builder.py
|   |   |   | workspace_builder.py
|   |   | etl/
|   |   |   | __init__.py
|   |   |   | loader.py
|   |   |   | connectors/
|   |   |   |   | __init__.py
|   |   |   |   | base_connector.py
|   |   |   |   | factory.py
|   |   |   |   | mongo_connector.py
|   |   |   |   | sql_connector.py
|   |   |   | sandbox/
|   |   |   |   | __init__.py
|   |   |   |   | executor.py
|   |   |   |   | sandbox_runner.py
|   |   | paths/
|   |   |   | __init__.py
|   |   |   | analytical.py
|   |   |   | metadata.py
|   |   |   | plot.py
|   |   |   | semantic.py
|   |   |   | sql_agent.py
|   | routers/
|   |   | __init__.py
|   |   | admin.py
|   |   | auth.py
|   |   | boards.py
|   |   | categories.py
|   |   | chat.py
|   |   | dashboard.py
|   |   | etl.py
|   |   | export.py
|   |   | files.py
|   |   | health.py
|   |   | langfuse_share.py
|   |   | preview.py
|   |   | projects.py
|   |   | query.py
|   |   | workspace.py
|   | schemas/
|   |   | __init__.py
|   |   | auth.py
|   |   | boards.py
|   |   | categories.py
|   |   | chat.py
|   |   | dashboard.py
|   |   | etl.py
|   |   | files.py
```

```

├── projects.py
├── query.py
├── workspace.py
├── services/
│   ├── __init__.py
│   ├── langfuse_sso.py
│   └── progress_manager.py
└── utils/
    ├── __init__.py
    ├── audit.py
    └── logging.py

```

main.py

The backend application bootstrap file.

Responsibilities:

- create FastAPI app
- attach lifespan startup/shutdown logic
- initialize database and supporting services
- register all routers

app/config.py

Global runtime configuration and service initialization.

Responsibilities:

- read environment variables
- initialize Redis connectivity
- configure Langfuse-related setup
- expose shared config needed across the backend

app/dependencies.py

Shared request-time dependency helpers.

Responsibilities:

- resolve a file identifier into the correct source
- infer whether a request is file mode or database mode
- enforce access permissions
- load the correct agent for the request

app/core/

This is the main backend logic layer.

Important files:

- `agent.py` Legacy `HybridAgent` orchestration for file-centric ingestion and querying.
- `auth.py` JWT, password hashing, RBAC, and user management helpers.
- `cache.py` Redis semantic cache operations.
- `database.py` `DatabaseManager`, connection pooling, and schema bootstrap.
- `embeddings.py` Embedding utilities.
- `file_ops.py` File delete/restore/hard-delete lifecycle helpers.
- `formatters.py` Result shaping for dashboard/query responses.
- `ingestion.py` File ingestion pipeline for Excel/CSV.
- `llm.py` Shared LLM helpers.
- `routing.py` Legacy query intent routing and cache coordination.
- `schema_utils.py` Workspace schema naming and schema-related helpers.

app/core/agents/

Workspace-focused AI modules.

- `profiler.py` Profiles workspace tables, generates descriptions, and stores semantic embeddings.
- `workspace_sql.py` Main workspace SQL agent for chat, reports, widgets, and dashboards.

app/core/dashboard/

Dashboard generation modules.

- `kpi_builder.py` Legacy file/project/board dashboard generation.
- `workspace_builder.py` Workspace-native dashboard generation, including streaming and multi-table logic.

app/core/etl/

Code-first ETL pipeline engine.

- `loader.py` Loads transformed parquet outputs into PostgreSQL workspace schemas.

app/core/etl/connectors/

Source-system connection layer.

- `base_connector.py` Abstract connector contract.
- `factory.py` Connector selection logic.
- `sql_connector.py` MySQL/PostgreSQL extraction and preview logic.
- `mongo_connector.py` MongoDB extraction and flattening logic.

app/core/etl/sandbox/

Transform execution sandbox.

- `executor.py` Runs transform scripts safely in subprocesses.
- `sandbox_runner.py` Lower-level sandbox runner support.

app/core/paths/

Legacy query execution paths chosen after intent classification.

- `analytical.py`
- `metadata.py`
- `plot.py`
- `semantic.py`
- `sql_agent.py`

Each file corresponds to one style of answer the old file-based AI system can produce.

app/routers/

HTTP API layer.

Each router file groups endpoints by product feature:

- `auth.py` login, register, password
- `admin.py` admin user management
- `files.py` upload, file metadata, trash, chunks
- `preview.py` preview and statistics
- `query.py` legacy natural-language file queries
- `dashboard.py` legacy dashboard generation and widget APIs
- `projects.py` project dashboards and active-file switching
- `boards.py` board management and board sharing/publishing
- `chat.py` legacy chat history
- `categories.py` saved question taxonomy
- `etl.py` source connection, dry-run, execute, sync, ETL editing
- `workspace.py` workspace CRUD, semantic layer, chat, report, and dashboard APIs
- `langfuse_share.py` public/shared endpoints and Langfuse token/SSO helpers
- `export.py` export support
- `health.py` health and info endpoints

app/schemas/

Pydantic request and response models organized by feature.

Why this folder matters:

- it tells you expected payload shapes
- it documents response contracts
- it is often the fastest way to verify what an endpoint accepts

app/services/

Small service helpers used across features.

- `langfuse_sso.py` Langfuse login/bootstrap support.
- `progress_manager.py` Upload progress tracking for SSE updates.

app/utils/

Cross-cutting support code.

- `logging.py` central logger setup
- `audit.py` audit-related helpers

System View

The product has two major working modes and one public consumption mode.

- Legacy file mode: Upload Excel/CSV, preview it, chat with it, generate dashboards, organize it into projects and boards.
- Workspace mode: Connect external DBs, transform noisy source data, load curated outputs into isolated workspace schemas, then chat/report/dashboard over those workspaces.
- Shared mode: Open public dashboards, workspaces, and reports using share tokens.

At runtime the system usually follows this shape:

1. A Vue page function reacts to a user action.
2. A service method in `excelApi.ts` or `workspaceApi.ts` sends a request.
3. A FastAPI router handles the HTTP request.
4. Core backend modules perform ingestion, routing, ETL, SQL generation, dashboard generation, or persistence.
5. PostgreSQL stores the result or metadata.
6. The API returns JSON, SSE, or NDJSON.
7. The frontend updates local state or Pinia and renders the UI.

Flow 1: User Login And Authenticated App Startup

Frontend start point

- `excel-ai-client/src/pages/login/index.vue`
- `excel-ai-client/src/store/login.ts`
- `excel-ai-client/src/services/excelApi.ts`
- `excel-ai-client/src/router/router.ts`

End-to-end flow

1. The user submits the login form in `pages/login/index.vue`.
2. `useLoginStore.login()` sends `POST /api/auth/login`.
3. Payload sent:

```
{
  "username": "analyst1",
  "password": "secret"
}
```

4. Backend route: `excel-ai-server-api/app/routers/auth.py:login()`.
5. `login()` calls `UserManager.get_user_by_login()` in `app/core/auth.py`.
6. `verify_password()` checks the stored hash.
7. `create_access_token()` creates the JWT.
8. Response returned to frontend:

```
{
  "access_token": "<jwt>",
  "token_type": "bearer",
  "user": {
    "id": "user-id",
    "username": "analyst1",
    "email": "analyst@org.com",
    "role": "user",
    "permissions": []
  }
}
```

9. The store saves the token in `localStorage`.
10. `excelApi.ts` request interceptor starts attaching `Authorization: Bearer <token>` to later calls.
11. `router.ts` now allows access to `/app/*`.

Related calls in the same auth journey

- If the user is new, the frontend can send `POST /api/auth/register` with a payload like:

```
{
  "username": "analyst1",
```

```

"email": "analyst@org.com",
"password": "secret",
"role": "user"
}

```

Backend route: `register()` in `auth.py`.

- When the app refreshes, `GET /api/auth/me` is used to reload current user info.
- When user changes password, `excelApi.changePassword()` sends:

```

{
  "current_password": "old-secret",
  "new_password": "new-secret"
}

```

to `POST /api/auth/change-password`, handled by `change_password()` in `auth.py`.

- When the user logs out, frontend can send `POST /api/auth/logout`, handled by `logout()` in `auth.py`.

Admin APIs in the same auth module

Administrative user management lives in `excel-ai-server-api/app/routers/admin.py`.

- `GET /api/admin/users` Returns all users for admin screens.
- `POST /api/admin/users` Payload example:

```

{
  "email": "new.user@org.com",
  "username": "newuser",
  "password": "secret",
  "role": "user"
}

```

- `PATCH /api/admin/users/{email}` Updates an existing user's profile, role, or allowed access.

Flow 2: Upload Excel/CSV And Register It As A Queryable File

Frontend start point

- `excel-ai-client/src/pages/upload/index.vue`
- `excel-ai-client/src/pages/documents/index.vue`
- `excel-ai-client/src/services/excelApi.ts`
- `excel-ai-client/src/store/documents.ts`

Main frontend functions

- `handleFileUpload()`
- `excelFileAPI.uploadFileWithProgress()`
- `checkForSavedQuestions()`
- `fetchFiles()`

End-to-end flow

1. The user chooses a file in `pages/upload/index.vue`.
2. `handleFileUpload()` calls `excelFileAPI.uploadFileWithProgress(file, onProgress)`.
3. That frontend helper first creates a client-side `file_uuid`.
4. Before the actual upload, it opens:

- `GET /api/files/upload-progress/{file_uuid}`

through `EventSource`, so the UI can receive progress stages.

5. It then sends the actual file to:

- `POST /api/files/upload`

using `multipart/form-data`.

6. Form payload includes:

- `file`
- `file_uuid`

- optional `project_id`
 - optional `subproject_id`
7. Backend route: `excel-ai-server-api/app/routers/files.py:upload_excel_file()`.
8. `upload_excel_file()` creates a progress callback via `app/services/progress_manager.py:create_progress_callback()`.
9. It constructs `HybridAgent(...)` from `app/core/agent.py`.
10. `HybridAgent` uses `IngestionMixin._ingest_file()` in `app/core/ingestion.py`.
11. Inside `_ingest_file()` the backend:
- reads the uploaded Excel/CSV
 - calls `_clean_dataframe()`
 - calls `_smart_parse_dates()`
 - calls `_generate_metadata_profile()`
 - creates row text for embeddings
 - creates PostgreSQL table storage
 - inserts rows
 - stores metadata in file registry
 - prepares the file for later query and dashboarding
12. While that is happening, the SSE endpoint returns progress events like:

```
{
  "stage": "embedding",
  "current": 40,
  "total": 100,
  "message": "Creating embeddings"
}
```

13. Final HTTP response from `POST /api/files/upload`:

```
{
  "status": "success",
  "message": "File uploaded successfully",
  "file_uuid": "uuid",
  "filename": "sales.xlsx",
  "table_name": "sales_2026",
  "rows": 1200,
  "columns": ["Month", "Region", "Sales"]
}
```

14. Frontend stores that in `fileWaitingForProcess`.
15. The upload page then calls `checkForSavedQuestions(file_uuid)`.
16. That triggers:

- `GET /api/questions/list/{file_uuid}`

handled by `list_saved_questions_for_file()` in `app/routers/categories.py`.

17. If saved questions exist, the UI offers replay; otherwise the user is routed to chat or documents.
18. The explorer later refreshes through `useDocumentsStore.fetchFiles()`, which calls:

- `GET /api/files`

handled by `list_files()` in `app/routers/files.py`.

Related file-management actions in the same module

When the user reorganizes or manages uploaded files, these calls happen in place:

- `PATCH /api/files/{file_uuid}/move` Payload:

```
{
  "target_folder_id": "project-id-or-null",
  "target_subfolder_id": "subproject-id-or-null"
}
```

Backend route: `move_file()` in `files.py`.

- `PATCH /api/files/{file_uuid}/metadata` Payload:

```
{
  "is_pinned": true,
}
```

```

    "is_favorite": false,
    "tags": ["finance", "monthly"]
  }

```

Backend route: `update_file_metadata()` in `files.py`.

- `GET /api/files/{file_identifier}` Used by `excelApi.getFileInfo(fileUuidOrFilename)` when the UI needs file metadata, columns, row count, and profile information.
- `GET /api/files/{file_identifier}/chunks` Used by `excelApi.getFileChunks(fileUuid, page, limit)` when the UI needs chunk-level content, row paging, or embedded text inspection.
- `DELETE /api/files/{file_identifier}` Backend route: `delete_file()` which uses soft-delete helpers in `app/core/file_ops.py`.
- `POST /api/files/{file_identifier}/restore` Backend route: `restore_file()`.
- `DELETE /api/files/{file_identifier}/permanent` Backend route: `permanent_delete_file()`.
- `GET /api/files/trash` Used by `excelApi.listTrashFiles()` to render recycle-bin views.
- `POST /api/files/bulk-move` Payload:

```

{
  "file_ids": ["file-a", "file-b"],
  "target_folder_id": "project-id-or-null",
  "target_subfolder_id": "subproject-id-or-null"
}

```

Backend route: `bulk_move_files()` in `files.py`.

- `DELETE /api/files/trash/empty` Used when the user empties the recycle bin in one action.

Flow 3: Upload A Multi-Sheet Workbook And Extract Selected Sheets

Frontend start point

- `excel-ai-client/src/pages/documents/index.vue`
- `excel-ai-client/src/services/excelApi.ts`

End-to-end flow

1. The user uploads a workbook that has multiple sheets.
2. Frontend calls:

- `POST /api/files/upload-raw`

through `excelApi.uploadRaw(file)`.

3. Payload is `multipart/form-data` with file.
4. Backend route: `upload_raw_file()` in `app/routers/files.py`.
5. Response:

```

{
  "temp_id": "temp-upload-id",
  "filename": "multi-sheet.xlsx",
  "sheets": ["Jan", "Feb", "Mar"]
}

```

6. The UI shows available sheets and lets the user choose which to ingest.
7. For each selected sheet, frontend calls:

- `POST /api/files/extract-sheet`

through `excelApi.extractSheet(req)`.

8. Payload:

```

{
  "temp_id": "temp-upload-id",
  "sheet_name": "Jan",
  "existing_group_id": "optional-group-id",
  "project_id": "optional-project-id",
}

```

```
"subproject_id": "optional-subproject-id"
}
```

9. Backend route: `extract_sheet()` in `files.py`.
10. It creates or reuses a workbook group entry.
11. It again builds a `HybridAgent` and runs ingestion for the specific sheet.
12. Response:

```
{
  "status": "success",
  "file_uuid": "new-sheet-file-id",
  "group_id": "group-id",
  "sheet_name": "Jan",
  "filename": "multi-sheet.xlsx",
  "table_name": "multi_sheet_jan",
  "rows": 500,
  "columns": ["Date", "Amount"]
}
```

13. The frontend stores `group_id` so later extracted sheets stay logically connected.
14. When the UI wants to show grouped workbook parents, it calls:

- `GET /api/files/groups`

handled by `list_file_groups()` in `files.py`.

Flow 4: Preview A File, Inspect Statistics, And Reuse Saved Questions

Frontend start point

- `excel-ai-client/src/pages/documents/index.vue`
- `excel-ai-client/src/pages/upload/index.vue`
- `excel-ai-client/src/services/excelApi.ts`

End-to-end flow: preview data

1. The user opens a preview action from the documents UI.
2. Frontend calls:

- `GET /api/preview/{file_identifier}`

through `excelApi.previewFile(fileId, rows)`.

3. Request carries:

- route param: `file_identifier`
- query param: `rows`

4. Backend route: `preview_file()` in `app/routers/preview.py`.
5. It resolves the file through the shared resolver utilities.
6. Response:

```
{
  "status": "success",
  "filename": "sales.xlsx",
  "total_rows": 1200,
  "preview_rows": 10,
  "columns": ["Month", "Region", "Sales"],
  "data": [
    {"Month": "Jan", "Region": "West", "Sales": 1200}
  ]
}
```

7. The frontend renders the modal/table preview.

End-to-end flow: statistics

1. If the UI asks for summary statistics, it calls:

- `GET /api/statistics/{file_identifier}`

2. Backend route: `get_statistics()` in `preview.py`.
3. The response is used to populate higher-level data summaries in the UI.

End-to-end flow: save a reusable question

1. User saves a question for later reuse.
2. Frontend calls:

- `POST /api/questions/save`

through `excelApi.saveQuestion(...)`.

3. Payload:

```
{
  "question_text": "Show total sales by region",
  "question_category": "Generic",
  "file_uuid": "file-id"
}
```

4. Backend route: `save_question()` in `app/routers/categories.py`.
5. The response is stored and later displayed in saved-question lists.

Other saved-question and category APIs in the same module

- `GET /api/categories` Used by `excelApi.listCategories()` to populate category dropdowns or management views.
- `PUT /api/categories/rename` Payload example:

```
{
  "old_name": "Generic",
  "new_name": "Executive"
}
```

Backend route: `rename_category()` in `categories.py`.

- `DELETE /api/categories/{category_name}` Removes a category from the saved-question system.
- `GET /api/questions/list` Returns all saved questions visible in the current user scope, not just those for one file.
- `PUT /api/questions/{question_id}` Payload example:

```
{
  "question_text": "Updated question",
  "question_category": "Executive"
}
```

Backend route: `update_saved_question()`.

- `DELETE /api/questions/{question_id}` Removes one saved question from the library.

End-to-end flow: replay saved questions in batch

1. After upload, the UI may offer auto-replay.
2. `runAutoReplay()` calls:

- `POST /api/query/batch`

3. Payload:

```
{
  "file_uuid": "file-id",
  "questions": [
    "Show total sales by region",
    "What is the trend month over month?"
  ],
  "session_id": "optional-session-id"
}
```

4. Backend route: `batch_execute_queries()` in `app/routers/query.py`.
5. That route repeatedly executes the legacy query pipeline for each question.
6. The upload page then routes the user into the chat page with a file context.

Flow 5: Ask A Natural-Language Question Over An Uploaded File

Frontend start point

- `excel-ai-client/src/pages/chat/index.vue`
- `excel-ai-client/src/services/excelApi.ts`

Main frontend functions

- `sendMessage()`
- `runQuery(userMessage, useCache, addUserBubble)`
- `bypassCache()`

End-to-end flow

1. The user types a question in the legacy chat page.
2. `sendMessage()` calls `runQuery(text, true, true)`.
3. `runQuery()` first ensures a session exists for tracing and chat continuity.
4. It determines `sourceType`:

- `file` for normal file mode
- `database` if route/tags indicate ETL-backed file-like mode

5. Frontend sends:

- `POST /api/query`

through `excelApi.queryFile(...)`.

6. Payload:

```
{
  "file_uuid": "file-id",
  "query": "Show month over month sales trend",
  "use_cache": true,
  "session_id": "langfuse-session-id",
  "source_type": "file"
}
```

7. Backend route: `excel-ai-server-api/app/routers/query.py:analyze_query()`.

8. `analyze_query()` uses shared helpers from `app/dependencies.py`:

- `infer_source_type()`
- `resolve_file_identifier()`
- `enforce_file_access()`
- `get_agent()` or `get_or_load_agent()`

9. That loads a `HybridAgent`.

10. `HybridAgent` uses routing logic in `app/core/routing.py`.

11. First it checks semantic cache via:

- `_normalize_query_with_llm()`
- `check_cache()`
- Redis semantic cache methods in `app/core/cache.py`

12. If there is no cache hit, `decide_intent()` chooses the execution path.

13. The question is then routed to one of these backend functions:

- `run_metadata_path()` in `app/core/paths/metadata.py`
- `run_analytical_path()` in `app/core/paths/analytical.py`
- `run_plot_path()` in `app/core/paths/plot.py`
- `run_sql_agent_path()` in `app/core/paths/sql_agent.py`
- `run_semantic_path()` in `app/core/paths/semantic.py`

14. Final response back to frontend might look like:

```
{
  "status": "success",
  "data": "{\"chart_type\":\"line\",\"data\":...}\",
  "query_type": "PLOT",
  "cache_hit": false,
  "trace_id": "trace-id",
  "trace_url": "https://langfuse/trace/..",
  "session_id": "session-id",
  "session_url": "https://langfuse/session/.."
}
```

15. Frontend then interprets the response:

- if `query_type` is plot, it parses chart JSON into `chartData`
- if `data` is table JSON, it extracts `tableData`
- otherwise it shows the answer as assistant text

16. After showing the answer, the chat page persists history by calling:

- `POST /api/chat/{file_uuid}`

once for the user message and once for the assistant result.

17. Example assistant save payload:

```
{
  "role": "assistant",
  "content": "Generated line chart",
  "query_type": "PLOT",
  "cache_hit": false,
  "response_time": 1.4,
  "metadata": {
    "sourceQuery": "Show month over month sales trend",
    "traceId": "trace-id",
    "traceUrl": "https://langfuse/trace/..",
    "chartData": {}
  }
}
```

18. Backend route: `save_chat_message()` in `app/routers/chat.py`.

19. Later, when the page loads history, it calls:

- `GET /api/chat/{file_uuid}`

handled by `get_chat_history()` in `chat.py`.

20. If the user clicks “bypass cache”, the same flow repeats but with:

```
{
  "use_cache": false
}
```

so the backend skips semantic cache reuse.

Other legacy chat-history APIs in the same module

- `DELETE /api/chat/{file_uuid}` Clears all stored chat messages for a file. Used by `excelApi.clearChatHistory(fileUuid)`.
- `POST /api/chat/{file_uuid}/soft_delete` Payload example:

```
{
  "message_id": 42
}
```

Marks one history item as removed without deleting the whole conversation.

Flow 6: Generate A Legacy File Dashboard

Frontend start point

- `excel-ai-client/src/pages/dashboard/SmartDashboard.vue`
- `excel-ai-client/src/services/excelApi.ts`

End-to-end flow

1. `SmartDashboard.vue` opens in file mode.
2. It calls:
 - `GET /api/dashboard/{file_identifier}`

through `excelApi.generateDashboard(fileUuid, regenerate, sessionId, mode, sourceType)`.

3. Query params sent may include:

```
{
  "regenerate": false,
  "session_id": "dashboard-session-id",
  "mode": "file",
  "source_type": "file"
}
```

4. Backend route: `generate_dashboard()` in `app/routers/dashboard.py`.
5. The route resolves the source and delegates to dashboard generation logic.
6. Core builder: `app/core/dashboard/kpi_builder.py:generate_kpi_dashboard()`.
7. That builder identifies useful metrics/dimensions and creates widget definitions.
8. Response returns dashboard payload containing widgets, layout hints, and metadata.
9. `SmartDashboard.vue` maps widgets into GridStack items and chart components.

Regenerate with requirements

1. The user customizes dashboard generation.
2. Frontend sends:

- `POST /api/dashboard/{file_identifier}/generate`

3. Payload:

```
{
  "user_requirements": "Focus on revenue, region, and month over month change",
  "session_id": "dashboard-session-id",
  "mode": "file",
  "source_type": "file"
}
```

4. Backend route: `generate_dashboard_custom()` in `dashboard.py`.
5. It uses the same builder logic but with user constraints applied.

Add one widget to the dashboard

1. User asks for one more widget inside the dashboard studio.
2. Frontend sends:

- `POST /api/dashboard/{file_identifier}/widget`

3. Payload:

```
{
  "query": "Show top 5 products by revenue",
  "session_id": "dashboard-session-id",
  "widget_type_hint": "chart",
  "chart_type_hint": "bar",
  "filter_context_hint": {
    "Region": "West"
  },
  "mode": "file",
  "source_type": "file"
}
```

4. Backend route: `generate_dashboard_widget()` in `dashboard.py`.
5. Builder function: `generate_single_widget()` in `kpi_builder.py`.
6. Response returns one widget object.
7. The frontend appends it to dashboard state.

Persist widget edits

1. After moving/removing/reordering widgets, the dashboard calls:

- `POST /api/dashboard/{file_identifier}/update-widgets`

2. Payload:

```
{
  "widgets": [],
  "session_id": "dashboard-session-id",
}
```

```
"studio": {}
}
```

3. Backend route: `update_dashboard_widgets()` in `dashboard.py`.
4. Response confirms updated widget state, and the frontend keeps local dashboard state synchronized.

Template-clone and export APIs in the same dashboard module

- `POST /api/dashboard/{file_identifier}/clone-template-widgets` Payload example:

```
{
  "template_widgets": [],
  "session_id": "dashboard-session-id",
  "mode": "file"
}
```

Backend route: `clone_template_widgets()` in `dashboard.py`. It regenerates saved template widgets for a new file while preserving the layout pattern.

- `POST /api/export/{file_identifier}` Used by export features when dashboard/file data is downloaded in formats like CSV or JSON. Backend route: `export_file()` in `app/routers/export.py`.

Flow 7: Cross-Filter And Chart-Builder Inside Legacy Dashboard

Frontend start point

- `excel-ai-client/src/pages/dashboard/SmartDashboard.vue`
- `excel-ai-client/src/services/excelApi.ts`

End-to-end flow: cross-filter

1. User clicks a chart segment or a filterable value.
2. Frontend sends:

- `POST /api/dashboard/{file_identifier}/filter`

3. Payload:

```
{
  "column": "Region",
  "value": "West",
  "session_id": "dashboard-session-id",
  "mode": "file",
  "source_type": "file"
}
```

4. Backend route: `filter_dashboard()` in `dashboard.py`.
5. Core builder function: `generate_filtered_dashboard()` in `kpi_builder.py`.
6. Response returns a dashboard rebuilt in filtered context.
7. `SmartDashboard.vue` replaces widgets with filtered widgets.

End-to-end flow: chart builder

1. The dashboard opens a manual chart-builder panel.
2. It first loads schema using:

- `GET /api/dashboard/{file_identifier}/schema`

3. Backend route: `get_dashboard_schema()` in `dashboard.py`.
4. Core builder function: `get_chart_schema()` in `kpi_builder.py`.
5. Response includes dimensions and measures that the UI uses to populate dropdowns.
6. When the user selects chart options, frontend sends:

- `POST /api/dashboard/{file_identifier}/chart-builder`

7. Payload:

```
{
  "chart_type": "bar",
  "dimension": "Region",
  "measure": "Sales",
  "aggregation": "sum",
  "mode": "file",
}
```

```
"source_type": "file"
}
```

- Backend route: `generate_custom_chart()` in `dashboard.py`.
- Core builder function: `generate_custom_chart_widget()` in `kpi_builder.py`.
- Response returns one deterministic chart widget.
- The UI renders it immediately.

Flow 8: Compare Two Files In Legacy Dashboard

Frontend start point

- `excel-ai-client/src/pages/dashboard/SmartDashboard.vue`
- `excel-ai-client/src/services/excelApi.ts`

End-to-end flow: split compare

- The user selects another file for compare mode.
- Frontend sends:

- `POST /api/dashboard/clone-widgets`

- Payload:

```
{
  "base_file_id": "file-a",
  "target_file_id": "file-b",
  "session_id": "compare-session-id"
}
```

- Backend route: `clone_widgets_for_compare()` in `dashboard.py`.
- Backend regenerates equivalent widgets for the second file using the base dashboard as a blueprint.
- Response returns the compare dashboard payload.
- `SmartDashboard.vue` shows side-by-side dashboards.

End-to-end flow: unified comparison

- If the user chooses unified compare, frontend sends:

- `POST /api/dashboard/compare-unified`

- Payload:

```
{
  "base_file_id": "file-a",
  "compare_file_id": "file-b",
  "session_id": "compare-session-id"
}
```

- Backend route: `compare_unified()` in `dashboard.py`.
- Core builder function: `generate_unified_comparison()` in `kpi_builder.py`.
- Response includes merged compare widgets and dataset metadata.
- The UI swaps into one combined comparison view.

Flow 9: Organize Files Into Projects And Use Project Dashboards

Frontend start point

- `excel-ai-client/src/store/documents.ts`
- `excel-ai-client/src/pages/documents/index.vue`
- `excel-ai-client/src/pages/dashboard/SmartDashboard.vue`
- `excel-ai-client/src/services/excelApi.ts`

End-to-end flow: create project and subproject

- Frontend creates a project by sending:

- `POST /api/projects`

- Payload:

```
{
  "name": "Q1 Sales",
  "color": "#2563eb",
  "is_dashboard": true,
  "source_file_uuid": "optional-file-id"
}
```

3. Backend route: `create_project()` in `app/routers/projects.py`.

4. The store refreshes by calling:

- `GET /api/projects`

handled by `list_projects()`.

5. For subprojects, frontend sends:

- `POST /api/projects/{project_id}/subprojects`

with:

```
{
  "name": "North Region"
}
```

6. Backend route: `create_subproject()`.

Other project-management APIs in the same module

- `PUT /api/projects/{project_id}` Payload example:

```
{
  "name": "Updated Project Name",
  "color": "#0f766e"
}
```

Backend route: `update_project()` in `projects.py`.

- `PUT /api/projects/{project_id}/subprojects/{subproject_id}` Payload example:

```
{
  "name": "Updated Subproject Name"
}
```

Backend route: `update_subproject()`.

- `DELETE /api/projects/{project_id}` Removes the project container.
- `DELETE /api/projects/{project_id}/subprojects/{subproject_id}` Removes one subproject.

End-to-end flow: open project dashboard

1. Frontend loads:

- `GET /api/projects/{project_id}/dashboard`

through `excelApi.getProjectDashboard(projectId)`.

2. Backend route: `get_project_dashboard()` in `projects.py`.

3. Response includes:

```
{
  "project": {},
  "files": [],
  "dashboard_data": {}
}
```

4. `SmartDashboard.vue` uses `project`, `files`, and `dashboard_data` to render project context.

End-to-end flow: switch active file in project dashboard

1. User changes the file attached to the project dashboard.
2. Frontend may send normal JSON request:

- `PUT /api/projects/{project_id}/active-file`

3. Payload:

```
{
  "file_uuid": "new-file-id"
}
```

4. Backend route: `update_active_file()` in `projects.py`.
5. Or, for stream-first UI updates, frontend may call:

- `PUT /api/projects/{project_id}/active-file-stream`

with the same payload.

6. Backend route: `update_project_active_file_stream()`.
7. Stream response sends NDJSON widgets one by one.
8. The UI updates incrementally while the dashboard rebuilds.

End-to-end flow: save project dashboard layout

1. Frontend sends:

- `POST /api/projects/{project_id}/dashboard/save`

2. Payload:

```
{
  "widgets": []
}
```

3. Backend route: `save_project_dashboard_layout()`.
4. The saved layout becomes the canonical project dashboard layout.

Flow 10: Use Insight Boards, Upload Compatible Files, And Publish Them

Frontend start point

- `excel-ai-client/src/store/boards.ts`
- `excel-ai-client/src/pages/dashboard/SmartDashboard.vue`
- `excel-ai-client/src/services/excelApi.ts`

End-to-end flow: create and load a board

1. Frontend creates a board using:

- `POST /api/boards`

2. Payload:

```
{
  "name": "Executive Review Board"
}
```

3. Backend route: `create_board()` in `app/routers/boards.py`.
4. Later it loads:

- `GET /api/boards/{board_id}/dashboard`

handled by `get_board_dashboard()`.

Other board list/delete APIs in the same module

- `GET /api/boards` Used by `excelApi.listBoards()` to render board pickers and board-management screens.
- `DELETE /api/boards/{board_id}` Used by `excelApi.deleteBoard(boardId)` to remove a board.

End-to-end flow: upload a board file

1. Frontend sends:

- `POST /api/boards/{board_id}/upload`
1. Payload is `multipart/form-data` with `file`.
 3. Backend route: `upload_board_file()` in `boards.py`.
 4. Response contains compatibility information:

```
{
  "status": "success",
  "file_uuid": "file-id",
  "filename": "board-source.xlsx",
  "rows": 400,
  "columns": ["Month", "Sales"],
  "compatible": true,
  "is_first_file": false,
  "missing_columns": [],
  "extra_columns": []
}
```

5. The UI uses this to decide whether templates or existing board widgets can be reused safely.

End-to-end flow: switch active file for board

1. Frontend calls:

- `PUT /api/boards/{board_id}/active-file`

or the stream variant:

- `PUT /api/boards/{board_id}/active-file-stream`

2. Payload:

```
{
  "file_uuid": "new-board-file-id",
  "session_id": "optional-session-id"
}
```

3. Backend routes:

- `update_board_active_file()`
- `update_board_active_file_stream()`

4. The board dashboard is regenerated using the selected file while preserving board context.

End-to-end flow: save board dashboard

1. Frontend sends:

- `POST /api/boards/{board_id}/dashboard/save`

2. Payload can be large and includes:

- `widgets`
- `screens`
- `active_screen_id`
- `active_theme`
- `screen_widgets`
- `file_screen_widgets`
- `file_screen_needs_generation`
- `file_screen_pending_templates`
- `screen_thumbnails`
- `file_screen_thumbnails`
- `design`

3. Backend route: `save_board_dashboard()` in `boards.py`.

4. That is why boards can preserve multi-screen dashboard studio state.

End-to-end flow: share or publish a board

1. When a user wants a live public board link, frontend calls:

- `POST /api/boards/{board_id}/share`

2. Payload may include:

```
{
  "widgets": [],
  "active_screen_id": "screen-id"
}
```

3. Backend route: `toggle_board_share()` in `boards.py`.

4. The backend:

- verifies ownership of the board
- toggles `is_shared`
- persists the current multi-screen studio state
- generates or reuses one public token per screen
- promotes the active screen token to the board-level `share_token`
- optionally stores the provided widget snapshot for the active screen

5. Typical response:

```
{
  "status": "success",
  "is_shared": true,
  "share_token": "screen-or-board-token",
  "share_tokens_by_screen": {
    "screen-overview": "token-a",
    "screen-detail": "token-b"
  },
  "active_screen_id": "screen-overview",
  "published_file_uuid": "file-id"
}
```

6. The frontend converts the returned token into a public `/share/{token}` URL and can expose per-screen links when the board has multiple screens.

End-to-end flow: freeze a published board snapshot

1. If the owner wants public viewers to see a frozen snapshot instead of the current live board state, frontend calls:

- `PUT /api/boards/{board_id}/publish`

2. Payload can be empty, or can explicitly publish the current in-memory widgets:

```
{
  "widgets": []
}
```

3. Backend route: `publish_board()` in `boards.py`.

4. The backend stores `published_file_uuid` and `published_widgets_json`.

5. Shared viewers will now read the frozen snapshot instead of the mutable live board state.

6. Response:

```
{
  "status": "success",
  "message": "Board published successfully",
  "published_file_uuid": "file-id"
}
```

End-to-end flow: unpublish a frozen board snapshot

1. To move the shared board back from frozen mode to live mode, frontend calls:

- `PUT /api/boards/{board_id}/unpublish`

2. Backend route: `unpublish_board()` in `boards.py`.

3. The backend clears the frozen snapshot columns and falls back to the latest saved board state.

4. Response:

```
{
  "status": "success",
  "message": "Published snapshot cleared - shared view is now live"
}
```

Flow 11: Open A Shared Public Dashboard And Cross-Filter It

Frontend start point

- `/share/:token`
- `excel-ai-client/src/services/excelApi.ts`

End-to-end flow: open a shared dashboard

1. A public user opens a tokenized dashboard route.
2. Frontend sends:

- `GET /api/share/{token}`

through `excelApi.getSharedDashboard(token)`.

3. Backend route: `get_shared_dashboard()` in `app/routers/langfuse_share.py`.
4. The backend resolves the token in this order:

- project share token
- board share token
- board screen-specific share token

5. If the token points to a board, the route serves either:

- the frozen published snapshot when `published_widgets_json` exists
- the current live saved board state when no frozen snapshot exists

6. Typical response:

```
{
  "project_id": "board-or-project-id",
  "project_name": "Executive Review Board",
  "file_uuid": "file-id",
  "dashboard_data": {
    "widgets": []
  },
  "screen_id": "screen-overview",
  "source": "board"
}
```

7. The public dashboard renders without normal authenticated app context.

Cross-filter in shared mode

1. User clicks a chart segment/value in the shared dashboard.
2. Frontend sends:

- `POST /api/share/{token}/filter`

3. Payload:

```
{
  "column": "Region",
  "value": "West",
  "session_id": "optional-session-id"
}
```

4. Backend route: `filter_shared_dashboard()` in `langfuse_share.py`.
5. The route resolves the underlying file from the share token, loads the correct project-aware or board-aware agent, then calls `generate_filtered_dashboard(filter_payload)`.
6. Response returns a filtered dashboard payload with refreshed widgets.
7. The public UI updates just like the authenticated dashboard, but in read-only/share mode.

Flow 12: Create A Workspace And Enter The ETL Wizard In Workspace Context

Frontend start point

- `excel-ai-client/src/pages/workspaces/index.vue`
- `excel-ai-client/src/pages/etl/index.vue`
- `excel-ai-client/src/services/workspaceApi.ts`

End-to-end flow

1. The user first creates a workspace from the workspace home page.
2. Frontend sends:

- `POST /api/workspaces`

3. Payload:

```
{
  "name": "Sales Analytics",
  "description": "Curated workspace for sales reporting"
}
```

4. Backend route: `create_workspace()` in `excel-ai-server-api/app/routers/workspace.py`.
5. The backend creates workspace metadata and the isolated target identity for later ETL loads.
6. Response:

```
{
  "workspace_id": "workspace-id",
  "name": "Sales Analytics",
  "schema_name": "ws_workspace_id",
  "description": "Curated workspace for sales reporting",
  "created_by": "user-id",
  "table_count": 0
}
```

7. The frontend then routes the user into ETL with workspace context, typically by opening a URL like:

- `/app/etl?workspace_id=<workspace-id>`

8. If the user is reopening ETL from an existing workspace, frontend may call:

- `GET /api/workspaces/{workspace_id}/etl-connection`

through `workspaceApi.getEtlConnection(workspaceId)`.

9. Backend route: `get_workspace_etl_connection()` in `workspace.py`.
10. That response tells the ETL page whether the workspace already has a linked `connection_id` or `job_id`, so the UI can reopen the correct pipeline instead of starting from scratch.
11. Typical response:

```
{
  "workspace_id": "workspace-id",
  "connection_id": "connection-id-or-null",
  "job_id": "job-id-or-null"
}
```

12. If `job_id` is present, the frontend can immediately continue the existing ETL lifecycle by reopening job detail through:

- `GET /api/etl/job/{job_id}/detail`

13. If both values are `null`, the ETL page falls back to the new-connection flow described in Flow 14.

Why this flow matters

The ETL wizard is not just a generic importer. It is usually opened in the context of one specific workspace, and that workspace later becomes the load target for curated data.

Flow 13: Browse, Rename, And Delete Existing Workspaces

Frontend start point

- `excel-ai-client/src/pages/workspaces/index.vue`
- `excel-ai-client/src/pages/workspaces/catalog/index.vue`
- `excel-ai-client/src/services/workspaceApi.ts`

Main frontend functions

- `listWorkspaces()`
- `updateWorkspace()`
- `deleteWorkspace()`

End-to-end flow: list workspaces

1. When the workspace home page loads, frontend sends:

- `GET /api/workspaces`

2. Backend route: `list_workspaces()` in `workspace.py`.

3. The backend reads all workspaces created by the current user and counts the tables currently loaded into each workspace.

4. Response:

```
{
  "workspaces": [
    {
      "workspace_id": "workspace-id",
      "name": "Sales Analytics",
      "schema_name": "ws_workspace_id",
      "description": "Curated workspace for sales reporting",
      "created_by": "user-id",
      "created_at": "2026-05-19T10:00:00",
      "table_count": 4
    }
  ]
}
```

5. The frontend uses this response to render workspace cards and navigation into ETL, catalog, chat, report, and dashboard views.

End-to-end flow: rename or update a workspace

1. When the user edits the workspace title or description, frontend sends:

- `PATCH /api/workspaces/{workspace_id}`

2. Payload:

```
{
  "name": "Updated Workspace Name",
  "description": "Updated description"
}
```

3. Backend route: `update_workspace()` in `workspace.py`.

4. The backend updates only the provided fields, keeps the schema identity stable, recomputes `table_count`, and returns the normalized workspace record.

5. Response:

```
{
  "workspace_id": "workspace-id",
  "name": "Updated Workspace Name",
  "schema_name": "ws_workspace_id",
  "description": "Updated description",
  "created_by": "user-id",
  "table_count": 4
}
```

End-to-end flow: delete a workspace

1. If the user removes a workspace, frontend sends:

- `DELETE /api/workspaces/{workspace_id}`

2. Backend route: `delete_workspace()` in `workspace.py`.

3. The backend deletes:

- workspace dashboard records
- semantic vectors, metrics, dimensions, and synonyms
- table and column metadata
- the workspace registry row
- the physical PostgreSQL schema for that workspace

4. Response:

```
{
  "message": "Workspace deleted",
  "workspace_id": "workspace-id"
}
```

5. The frontend removes the workspace from the list and returns the user to the workspace landing page.

Flow 14: Connect A Source Database In ETL Wizard And Preview Raw Tables

Frontend start point

- `excel-ai-client/src/pages/etl/index.vue`
- `excel-ai-client/src/services/excelApi.ts`

Main frontend functions

- `handleConnect()`
- `useSavedConnection()`
- `previewTable()`

End-to-end flow

1. User enters source DB connection info in the ETL page.
2. `handleConnect()` sends:

- `POST /api/etl/connect`

3. Payload:

```
{
  "db_type": "postgresql",
  "host": "localhost",
  "port": 5432,
  "username": "source_user",
  "password": "secret",
  "database": "sales_db",
  "auth_source": "admin",
  "connection_name": "Sales Warehouse"
}
```

4. Backend route: `etl_connect()` in `app/routers/etl.py`.
5. `etl_connect()` builds `ConnectionConfig` and calls `create_connector(config)`.
6. Source-specific connector then runs:

- `connect()`
- `test_connection()`
- `list_tables()`

7. Response:

```
{
  "connection_id": "conn-id",
  "db_type": "postgresql",
  "database": "sales_db",
  "tables": [
    {
      "name": "orders",
      "row_count": 123456,
      "columns": [{"name": "id", "type": "integer"}]
    }
  ],
  "message": "Connected successfully"
}
```

8. The UI stores `connectionId` and available tables.
9. It immediately previews a table by calling:

- `POST /api/etl/preview-table`

10. Payload:

```
{
  "connection_id": "conn-id",
  "table_name": "orders",
  "limit": 20
}
```

11. Backend route: `etl_preview_table()` in `etl.py`.
12. Connector function: `preview_table()` in `app/core/etl/connectors/sql_connector.py` or connector equivalent.
13. Response:

```
{
  "table_name": "orders",
  "rows": [],
  "total_rows": 20,
  "columns": ["id", "amount", "updated_at"]
}
```

14. The ETL page renders raw-source preview in the right-side panel.

Reuse saved connections

1. If the user chooses a saved connection, frontend sends:

- `POST /api/etl/connect`

with:

```
{
  "connection_id": "saved-connection-id"
}
```

2. Backend route rehydrates connection metadata and reconnects through `_get_or_restore_connector()`.

Connection-management APIs in the same ETL module

- `GET /api/etl/connections` Used by `loadSavedConnections()` through `excelApi.etlListConnections()` to populate saved source connections.
- `DELETE /api/etl/connections/{connection_id}` Used by `deleteSavedConnection(connId)` to remove a saved source connection.
- `GET /api/etl/datasets` Used by `excelApi.listETLDatasets()` or `getEtlDatasets()` to provide a grouped view of ETL-created datasets and jobs.

Flow 15: Dry-Run A Transform Script On Sample Source Data

Frontend start point

- `excel-ai-client/src/pages/etl/index.vue`
- `excel-ai-client/src/services/excelApi.ts`

Main frontend functions

- `buildExtractDatasets()`
- `handleDryRun()`

End-to-end flow

1. User chooses source tables or writes custom queries.
2. The ETL page builds dataset descriptors through `buildExtractDatasets()`.
3. If the user wants AI help generating transform code, frontend may first call:

- `POST /api/etl/generate-transform`

with:

```
{
  "prompt": "Clean orders, cast dates, and remove null customers",
  "tables": []
}
```

4. Backend route: `generate_transform()` in `etl.py`.

5. Response contains generated script text.

6. When user clicks dry-run, frontend sends:

- `POST /api/etl/dry-run`

7. Payload:

```
{
  "connection_id": "conn-id",
  "table_names": ["orders", "customers"],
  "datasets": [
    {
      "custom_query": "select * from orders where updated_at > {{LAST_SYNC_VALUE}}",
      "output_name": "recent_orders",
      "sync_column": "updated_at"
    }
  ],
  "transform_script": "def transform(tables): return tables"
}
```

8. Backend route: `etl_dry_run()` in `etl.py`.

9. Inside `etl_dry_run()`, the backend creates a temporary working area with:

- `work_dir`
- `input_dir = work_dir/input`
- `output_dir = work_dir/output`
- `script_path = work_dir/transform.py`

10. Before the transform script runs, the extract phase happens first.

11. `etl_dry_run()` calls:

- `connector.extract_to_parquet(...)`

12. If the source is SQL, this goes into `app/core/etl/connectors/sql_connector.py:extract_to_parquet()`.

13. That SQL connector executes source-side SQL using its SQLAlchemy engine and `pandas.read_sql(...)`.

14. The exact query shape depends on the ETL selection:

- for selected tables it builds queries like `SELECT * FROM <table> LIMIT 100`
- for custom query datasets it uses the user SQL, normalized by `_normalize_read_query()`, and enforces the sample limit

15. The extracted raw source data is written as parquet files into:

- `work_dir/input/*.parquet`

16. Those parquet files are not the final loaded data. They are temporary extracted inputs used by the transform stage.

17. After extraction, the backend writes the transform code into:

- `work_dir/transform.py`

18. Then `ETLExecutor.run_transform()` from `app/core/etl/sandbox/executor.py` loads the input parquet files, runs the user `transform()` function in a sandboxed subprocess, and writes transformed outputs into:

- `work_dir/output/*.parquet`

19. Only after that does the backend read the transformed parquet files back from `output_dir`, serialize preview rows, and prepare the API response.

20. Response:

```
{
  "success": true,
  "duration_seconds": 4.2,
  "output_tables": [
    {
      "name": "recent_orders",
      "row_count": 100,
      "columns": ["order_id", "customer_id", "amount"],
      "preview": []
    }
  ],
  "stdout": "script logs..."
}
```

21. The frontend displays output table previews, timing, stdout, and any transform errors.

Flow 16: Execute Full ETL Pipeline And Poll Job Progress

Frontend start point

- `excel-ai-client/src/pages/etl/index.vue`
- `excel-ai-client/src/services/excelApi.ts`

Main frontend functions

- `handleExecute()`
- `startPolling()`

End-to-end flow

1. The user confirms full ETL execution.
2. `handleExecute()` sends:

- `POST /api/etl/execute-job`

3. Payload:

```
{
  "job_id": "optional-existing-job-id",
  "connection_id": "conn-id",
  "table_names": ["orders"],
  "datasets": [],
  "transform_script": "def transform(tables): return tables",
  "target_table": "fact_orders",
  "pipeline_name": "Orders Pipeline",
  "sync_mode": "overwrite",
  "sync_column": "updated_at",
  "workspace_id": "workspace-id",
  "primary_keys": ["order_id"]
}
```

4. Backend route: `etl_execute_job()` in `etl.py`.
5. The route stores job metadata in `etl_system.etl_jobs`.
6. It schedules background pipeline execution through `_run_full_etl_pipeline(...)`.
7. Immediate response:

```
{
  "job_id": "job-id",
  "status": "pending",
  "message": "Job submitted successfully"
}
```

8. Frontend stores `jobId` and starts polling:

- `GET /api/etl/job/{job_id}`

through `etlJobStatus(jobId)`.

9. Backend route: `etl_job_status()` in `etl.py`.
10. While the job runs, the backend follows the same three-stage ETL structure as dry-run, but on full data instead of a sample.
11. Extract stage:

- `_run_full_etl_pipeline(...)` calls `connector.extract_to_parquet(...)`
- for SQL sources, `sql_connector.py:extract_to_parquet()` executes source SQL or table reads against the source database
- raw extracted datasets are written as parquet files into a temporary input folder for this job

12. Transform stage:

- the job writes the transform code to a temporary `transform.py`
- `ETLExecutor.run_transform()` reads the extracted parquet inputs
- transformed outputs are written into a temporary output parquet folder

13. Load stage:

- `ETLLoader.load_to_postgres(parquet_dir=output_dir, schema_name=...)` loads transformed parquet outputs into PostgreSQL
- the target schema is the workspace schema, typically derived through `get_workspace_schema_name(workspace_id)`
- so final curated tables land in a schema like `ws_<workspace_id>`

14. Metadata persistence stage:

- ETL job state is stored in `etl_system.etl_jobs`
- connection metadata lives in `etl_system.etl_connections`
- loaded-table visibility is reflected back into workspace metadata tables, so workspace catalog/chat/dashboard can see the new tables

15. Important architectural point:

- raw source data is temporarily saved as parquet in the job input folder
- transformed curated data is temporarily saved as parquet in the job output folder
- the permanent persisted analytics data is the final PostgreSQL table inside the workspace schema
- the temporary parquet working folders are used as ETL pipeline stages and then cleaned up

16. Poll response includes fields like:

```
{
  "job_id": "job-id",
  "status": "loading",
  "source_tables": [],
  "output_tables": [],
  "error_message": null,
  "started_at": "...",
  "completed_at": null,
  "target_table": "fact_orders",
  "sync_mode": "overwrite",
  "high_water_mark": null,
  "sync_column": "updated_at"
}
```

17. The ETL page keeps polling until `status` becomes `complete` or `failed`.

18. When complete, the UI tells the user the workspace data is now ready for chat/report/dashboard use.

Related ETL maintenance calls in the same lifecycle

- `GET /api/etl/connection/{connection_id}/jobs` Used by `etlGetConnectionJobs()` to load prior pipelines for a connection.
- `GET /api/etl/job/{job_id}/detail` Used by `etlGetJobDetail()` when reopening an existing pipeline to edit it.
- `PUT /api/etl/job/{job_id}` Used by `etlUpdateJob(payload)` when saving pipeline edits.
- `PATCH /api/etl/job/{job_id}/sync-config` Payload example:

```
{
  "sync_column": "updated_at",
  "sync_mode_type": "hwm"
}
```

Used to refine incremental sync behavior after the job exists.

Flow 17: Run Delta Sync Or Targeted Sync-Update On An Existing Pipeline

Frontend start point

- `excel-ai-client/src/pages/etl/index.vue`
- `excel-ai-client/src/services/excelApi.ts`

End-to-end flow: delta sync

1. User clicks sync for an existing pipeline.
2. Frontend sends:

- `POST /api/etl/job/{job_id}/sync`

3. Payload:

```
{
  "sync_column": "updated_at",
  "preview_only": true
}
```

4. Backend route: `etl_delta_sync()` in `etl.py`.

5. The route:

- reloads saved pipeline metadata
- determines source queries/tables
- replaces `{{LAST_SYNC_VALUE}}` if present
- looks up current max values in loaded workspace tables
- extracts only the delta
- reruns transform
- optionally previews or loads new rows

6. Preview response may contain:

```
{
  "success": true,
  "loaded_tables": [],
  "preview_tables": [],
  "total_new_rows": 125,
  "requires_confirmation": true,
  "message": "Found 125 new rows across 1 table(s). Review and continue to load."
}
```

7. The frontend uses this to show confirmation UI before actual load.

End-to-end flow: explicit sync-update

1. Frontend may instead send:

- `POST /api/etl/sync-update`

2. Payload:

```
{
  "connection_id": "conn-id",
  "workspace_id": "workspace-id",
  "target_table_name": "fact_orders",
  "table_names": ["orders"],
  "datasets": [],
  "transform_script": "def transform(tables): return tables"
}
```

3. Backend route: `etl_sync_update()` in `etl.py`.

4. The response tells the frontend what was loaded and how many rows were updated.

Flow 18: Open A Workspace And Load Its Metadata Catalog

Frontend start point

- `excel-ai-client/src/pages/workspaces/index.vue`
- `excel-ai-client/src/pages/workspaces/catalog/index.vue`
- `excel-ai-client/src/services/workspaceApi.ts`

End-to-end flow

1. After ETL load is complete, or when a user reopens an existing workspace, the frontend navigates to a workspace-specific page such as:

- `/app/workspaces/{workspace_id}`
- `/app/workspaces/{workspace_id}/catalog`

2. `pages/workspaces/catalog/index.vue:fetchDetails()` loads detailed workspace information.

3. Frontend sends:

- `GET /api/workspaces/{workspace_id}`

4. Backend route: `get_workspace_detail()`.

5. The response includes workspace metadata, table list, and nested column metadata for each known table.

6. Typical response shape:

```
{
  "workspace_id": "workspace-id",
  "name": "Sales Analytics",
  "schema_name": "ws_workspace_id",
  "description": "Curated workspace for sales reporting",
  "table_count": 2,
  "tables": [
    {
      "metadata_id": "meta-id",
      "table_name": "fact_orders",
      "schema_name": "ws_workspace_id",
      "description": "One row per order",
      "row_count": 12000,
      "columns": []
    }
  ]
}
```

7. The page then loads semantic layer with:

- `GET /api/workspaces/{workspace_id}/semantic`

8. Backend route: `get_semantic_layer()`.

9. Response contains metrics, dimensions, and synonyms.

10. The page also loads relationships with:

- `GET /api/workspaces/{workspace_id}/relationships`

11. Backend route: `list_workspace_relationships()`.

12. All three responses are merged by the page into one workspace-catalog UI.

End-to-end flow: load a table-only catalog view

1. If the UI needs a lighter table list without the full nested workspace payload, frontend sends:

- `GET /api/workspaces/{workspace_id}/tables`

2. Backend route: `list_workspace_tables()`.

3. Response shape:

```
{
  "workspace_id": "workspace-id",
  "tables": [
    {
      "metadata_id": "meta-id",
      "table_name": "fact_orders",
      "schema_name": "ws_workspace_id",
      "description": "One row per order",
      "row_count": 12000,
      "created_at": "2026-05-19T10:00:00"
    }
  ]
}
```

End-to-end flow: inspect column metadata for one table

1. When a user clicks a table in the catalog, frontend sends:

- `GET /api/workspaces/{workspace_id}/tables/{table_name}/columns`

2. Backend route: `list_table_columns()`.

3. Response shape:

```
{
  "workspace_id": "workspace-id",
  "table_name": "fact_orders",
  "columns": [
    {
      "column_id": "column-id",
      "column_name": "updated_at",
      "data_type": "timestamp",
      "description": "UTC timestamp when the order was last updated",
      "sample_values": [],
      "stats": {}
    }
  ]
}
```

End-to-end flow: curate table and column descriptions

1. To improve business context for later profiling and SQL generation, frontend can update a table description with:

- `PATCH /api/workspaces/{workspace_id}/tables/{table_name}/description`

2. Payload:

```
{
  "description": "One row per order transaction"
}
```

3. Backend route: `update_table_description()`.

4. Response:

```
{
  "message": "Description updated",
  "table_name": "fact_orders"
}
```

5. To refine a specific column description, frontend sends:

- `PATCH /api/workspaces/{workspace_id}/columns/{column_id}/description`

6. Payload:

```
{
  "description": "UTC timestamp when the order was last updated"
}
```

7. Backend route: `update_column_description()`.

8. Response:

```
{
  "message": "Column description updated",
  "column_id": "column-id"
}
```

Flow 19: Profile Workspace Tables And Build Semantic Context

Frontend start point

- `excel-ai-client/src/pages/workspaces/catalog/index.vue`
- `excel-ai-client/src/services/workspaceApi.ts`

Main frontend functions

- `handleProfileWorkspace()`
- `fetchDetails()`

End-to-end flow

1. The user clicks "Profile Workspace".
2. Frontend sends:

- `POST /api/workspaces/{workspace_id}/profile`

through `workspaceApi.profileWorkspace(workspaceId)`.

3. Backend route: `profile_workspace()` in `workspace.py`.
4. It delegates to `app/core/agents/profiler.py:profile_workspace()`.
5. That profiler walks workspace tables and columns, generates descriptions, and stores semantic embeddings.
6. For single-table profiling, frontend may call:

- `POST /api/workspaces/{workspace_id}/profile/{table_name}`

handled by `profile_table()`.

7. Once profiling finishes, frontend calls `fetchDetails()` again to reload:

- `GET /api/workspaces/{workspace_id}`
- `GET /api/workspaces/{workspace_id}/semantic`
- `GET /api/workspaces/{workspace_id}/relationships`

8. The refreshed metadata makes later SQL generation and dashboards better.

Flow 20: Define Relationships, Metrics, Dimensions, And Synonyms

Frontend start point

- `excel-ai-client/src/pages/workspaces/catalog/index.vue`
- `excel-ai-client/src/services/workspaceApi.ts`

End-to-end flow: add a relationship

1. User defines a join relationship in the relationships tab.
2. Frontend sends:

- `POST /api/workspaces/{workspace_id}/relationships`

3. Payload:

```
{
  "left_table": "fact_orders",
  "left_column": "customer_id",
  "right_table": "dim_customer",
  "right_column": "id",
  "relationship_type": "many_to_one"
}
```

4. Backend route: `create_workspace_relationship()` in `workspace.py`.
5. The frontend then refreshes relationship list with:

- `GET /api/workspaces/{workspace_id}/relationships`

End-to-end flow: create semantic entities

Metric creation:

1. Frontend sends:

- `POST /api/workspaces/{workspace_id}/semantic/metrics`

2. Payload:

```
{
  "name": "total_sales",
  "formula": "SUM(fact_orders.amount)",
  "description": "Total booked sales amount",
  "related_tables": ["fact_orders"]
}
```

3. Backend route: `create_metric()`.

Dimension creation:

1. Frontend sends:

- `POST /api/workspaces/{workspace_id}/semantic/dimensions`

2. Payload:

```
{
  "name": "region",
  "table_name": "dim_customer",
  "column_name": "region",
  "description": "Customer sales region",
  "dim_type": "categorical"
}
```

3. Backend route: `create_dimension()`.

Synonym creation:

1. Frontend sends:

- `POST /api/workspaces/{workspace_id}/semantic/synonyms`

2. Payload:

```
{
  "keyword": "revenue",
  "mapped_to": "total_sales",
  "mapped_type": "metric"
}
```

3. Backend route: `create_synonym()`.

4. After each create/delete operation, the page refreshes semantic layer using:

- `GET /api/workspaces/{workspace_id}/semantic`

This is the key loop that makes workspace AI more business-aware over time.

Delete APIs in the same semantic module

- `DELETE /api/workspaces/{workspace_id}/semantic/metrics/{metric_id}` Removes a semantic metric from the workspace model.
- `DELETE /api/workspaces/{workspace_id}/semantic/dimensions/{dimension_id}` Removes a semantic dimension.
- `DELETE /api/workspaces/{workspace_id}/semantic/synonyms/{synonym_id}` Removes a synonym mapping.
- `DELETE /api/workspaces/{workspace_id}/relationships/{relationship_id}` Removes a declared relationship between tables.

Flow 21: Ask Questions In Workspace Chat And Get SQL-Backed Answers

Frontend start point

- `excel-ai-client/src/pages/workspaces/chat/index.vue`
- `excel-ai-client/src/services/workspaceApi.ts`

Main frontend functions

- `sendMessage()`
- `loadChatHistory()`
- `openPinModal()`
- `handlePinToDashboard()`

End-to-end flow

1. User asks a question in workspace chat.
2. Frontend pushes a local user bubble and sends:

- `POST /api/workspaces/{workspace_id}/chat`

3. Payload:

```
{
  "question": "Show monthly sales trend by region",
  "session_id": "optional-session-id",
  "title": "Monthly Sales Trend"
}
```

4. Backend route: `workspace_chat()` in `app/routers/workspace.py`.
5. That route uses `WorkspaceSqlAgent` from `app/core/agents/workspace_sql.py`.
6. Important backend function chain:

- `chat()`
- `_retrieve_context()`
- semantic/table metadata lookup
- `_vector_search()`
- `_generate_sql()`
- `_validate_sql()`
- `_execute_sql()`
- `_regenerate_sql_from_error()` if needed

7. Response:

```
{
  "status": "success",
  "question": "Show monthly sales trend by region",
  "sql": "SELECT ...",
  "explanation": "Here are the results for your query",
  "columns": ["month", "region", "sales"],
  "data": [],
  "row_count": 36
}
```

8. The frontend renders:

- markdown explanation
- KPI cards if one row

- table view if many rows
- pin-to-dashboard option

Chat history in the same workspace flow

1. When the page opens, it calls:

- `GET /api/workspaces/{workspace_id}/chat/history`

2. Backend route: `get_chat_history()` in `workspace.py`.

3. If the user clears history, frontend sends:

- `DELETE /api/workspaces/{workspace_id}/chat/history`

handled by `clear_chat_history()`.

4. If one item is deleted, frontend sends:

- `DELETE /api/workspaces/{workspace_id}/chat/history/{message_id}`

handled by `delete_chat_history_item()`.

Flow 22: Generate Workspace Reports, Run Custom SQL, Summarize, And Share Them

Frontend start point

- `excel-ai-client/src/pages/workspaces/report/index.vue`
- `excel-ai-client/src/services/workspaceApi.ts`

Main frontend functions

- `generateReport()`
- `runHistoricalReport()`
- `forwardToAIRport()`
- `shareThisReport()`

End-to-end flow: AI report mode

1. In report page AI mode, user enters a question and optional title.
2. Frontend sends:

- `POST /api/workspaces/{workspace_id}/chat`

because the report page reuses the workspace chat SQL-generation pipeline.

3. Payload:

```
{
  "question": "Show top regions by sales",
  "title": "Top Regions"
}
```

4. Backend route: `workspace_chat()`.

5. The returned `ChatResponse` becomes `currentReport`.

End-to-end flow: custom SQL report mode

1. In SQL mode, frontend sends:

- `POST /api/workspaces/{workspace_id}/report/execute_custom_sql`

2. Payload:

```
{
  "title": "Custom SQL Report",
  "sql_query": "select region, sum(amount) as sales from fact_orders group by region"
}
```

3. Backend route: `execute_custom_sql()` in `workspace.py`.

4. Response is again a `ChatResponse`-style structure with `columns/data/row_count`.

5. The page renders the tabular result and enables CSV export.

End-to-end flow: AI summary for a report

1. When the user clicks AI summary, frontend sends:

- `POST /api/workspaces/{workspace_id}/report/summarize`

2. Payload:

```
{
  "question": "Top Regions",
  "sql_query": "select region, sum(amount) as sales from fact_orders group by region",
  "columns": ["region", "sales"],
  "data": [
    {"region": "West", "sales": 1200}
  ]
}
```

3. Backend route: `summarize_workspace_report()`.

4. Response:

```
{
  "status": "success",
  "summary": "West leads sales, followed by..."
}
```

5. The frontend renders markdown summary in the report page.

End-to-end flow: share a report snapshot

1. Frontend sends:

- `POST /api/workspaces/{workspace_id}/report/share`

2. Payload:

```
{
  "title": "Top Regions",
  "question": "Show top regions by sales",
  "sql_query": "select region, sum(amount) as sales from fact_orders group by region",
  "columns": ["region", "sales"],
  "data": [],
  "row_count": 10,
  "ai_summary": "West is leading sales..."
}
```

3. Backend route: `share_report_snapshot()` in `workspace.py`.

4. Response:

```
{
  "status": "success",
  "report_id": "report-id",
  "share_token": "token"
}
```

5. The frontend converts that token into a public `/shared/report/{token}` URL.

Flow 23: Generate And Maintain A Workspace Dashboard

Frontend start point

- `excel-ai-client/src/pages/workspaces/dashboard/index.vue`
- `excel-ai-client/src/pages/workspaces/chat/index.vue`
- `excel-ai-client/src/services/workspaceApi.ts`

Main frontend functions

- `load()`
- `refreshAllWidgets()`
- `refreshSingleWidget()`
- `changeTheme()`
- `onSlicerChange()`
- `clearSlicers()`

End-to-end flow: load workspace dashboard

1. The dashboard page opens.
2. Frontend sends:

- `GET /api/workspaces/{workspace_id}/dashboard`

through `workspaceApi.getDashboard(workspaceId, regenerate)`.

3. If regenerate is requested, query params include:

```
{
  "regenerate": true
}
```

4. Backend route: `get_workspace_dashboard()` in `workspace.py`.
5. Backend may delegate to dashboard-generation logic in:

- `WorkspaceSqlAgent.generate_workspace_dashboard()`
- `app/core/dashboard/workspace_builder.py`

6. Response:

```
{
  "status": "success",
  "workspace_id": "workspace-id",
  "dashboard_id": "dashboard-id",
  "dashboard_name": "Sales Dashboard",
  "layout_json": {},
  "widgets": []
}
```

7. The frontend maps widgets into GridStack and ECharts state.

End-to-end flow: enable or inspect workspace sharing

1. When the workspace owner wants to expose a public dashboard/chat/report experience, frontend sends:

- `POST /api/workspaces/{workspace_id}/share/toggle`

2. Backend route: `toggle_workspace_sharing()` in `workspace.py`.
3. The backend flips `is_shared`, generates a `share_token` if needed, and stores it on the workspace record.
4. Response:

```
{
  "status": "success",
  "workspace_id": "workspace-id",
  "is_shared": true,
  "share_token": "workspace-share-token"
}
```

5. The frontend then checks or reloads share state with:

- `GET /api/workspaces/{workspace_id}/share/status`

6. Backend route: `get_workspace_share_status()`.
7. Response:

```
{
  "workspace_id": "workspace-id",
  "is_shared": true,
  "share_token": "workspace-share-token"
}
```

8. The UI converts that token into a public `/shared/workspace/{token}` style route for consumers.

End-to-end flow: stream workspace dashboard generation

1. If the UI wants incremental widget loading, it calls:

- `GET /api/workspaces/{workspace_id}/dashboard/stream`

through `workspaceApi.streamDashboard(workspaceId, onWidget)`.

- Backend route: `stream_workspace_dashboard()`.
- The route streams NDJSON, one widget at a time.
- Frontend reads each line, parses it with `JSON.parse()`, and appends widgets incrementally.

End-to-end flow: save dashboard state

- After layout or widget changes, frontend sends:
 - `POST /api/workspaces/{workspace_id}/dashboard/save`
- Payload:

```
{
  "name": "Sales Dashboard",
  "layout_json": {},
  "widgets": []
}
```

- Backend route: `save_workspace_dashboard()`.
- This persists the workspace dashboard definition.

Flow 24: Pin A Workspace Chat Result, Generate Extra Widgets, Refresh Them, And Build Custom Charts

Frontend start point

- `excel-ai-client/src/pages/workspaces/chat/index.vue`
- `excel-ai-client/src/pages/workspaces/dashboard/index.vue`
- `excel-ai-client/src/services/workspaceApi.ts`

End-to-end flow: pin chat result to dashboard

- In workspace chat, user clicks pin.
- `handlePinToDashboard()` sends:
 - `POST /api/workspaces/{workspace_id}/dashboard/pin`
- Payload:

```
{
  "title": "Monthly Trend",
  "widget_type": "chart",
  "chart_type": "bar",
  "sql_query": "SELECT ...",
  "origin_question": "Show monthly trend",
  "config": {
    "source_table": "_chat",
    "columns": ["month", "sales"],
    "chartData": [],
    "row_count": 12
  }
}
```

- Backend route: `pin_widget_to_dashboard()` in `workspace.py`.
- The widget is persisted as part of workspace dashboard data.

End-to-end flow: generate an AI widget from dashboard page

- User asks for another widget from workspace dashboard UI.
- Frontend sends:
 - `POST /api/workspaces/{workspace_id}/dashboard/widget`
- Payload:

```
{
  "query": "Create a region-wise sales chart",
  "widget_type_hint": "chart",
  "chart_type_hint": "bar"
}
```

- Backend route: `generate_workspace_widget()`.
- Backend uses `WorkspaceSqlAgent.generate_single_widget()`.

6. Response returns one widget and the UI inserts it into dashboard state.

End-to-end flow: create a custom SQL widget

1. Frontend sends:

- `POST /api/workspaces/{workspace_id}/dashboard/widget/custom`

2. Payload:

```
{
  "title": "Top 10 customers",
  "sql_query": "select customer_name, sum(amount) as sales from fact_orders group by customer_name",
  "widget_type_hint": "chart",
  "chart_type_hint": "bar"
}
```

3. Backend route: `create_workspace_custom_widget()`.
4. Response returns a widget object ready for display.

End-to-end flow: refresh widgets with slicers

1. User changes slicer values in the dashboard UI.
2. `refreshAllWidgets()` loops through existing widgets and calls:

- `POST /api/workspaces/{workspace_id}/dashboard/widgets/{widget_id}/refresh`

3. Payload:

```
{
  "filters": {
    "Region": "West",
    "Year": "2026"
  }
}
```

4. Backend route: `refresh_workspace_widget()`.
5. Backend reruns widget data generation in filtered context.
6. The specific widget updates in place.

End-to-end flow: manual chart builder

1. The dashboard first loads schema using:

- `GET /api/workspaces/{workspace_id}/dashboard/schema`

2. Backend route: `get_workspace_chart_schema()`.

3. The UI uses returned tables/dimensions/measures to populate chart-builder controls.

4. User then sends:

- `POST /api/workspaces/{workspace_id}/dashboard/chart-builder`

5. Payload:

```
{
  "chart_type": "bar",
  "dimension": "region",
  "measure": "amount",
  "aggregation": "sum"
}
```

6. Backend route: `workspace_chart_builder()`.
7. Backend function: `WorkspaceSqlAgent.generate_custom_chart()`.
8. Response returns a chart widget which the UI renders directly.

Remove widget API in the same workspace-dashboard module

- `DELETE /api/workspaces/{workspace_id}/dashboard/widgets/{widget_id}` Used by `workspaceApi.removeWidget()` or `unpinWidget()` to remove one saved widget from the workspace dashboard.

Frontend start point

- `excel-ai-client/src/pages/workspaces/shared/SharedWorkspaceDashboard.vue`
- `excel-ai-client/src/pages/workspaces/shared/SharedWorkspaceChat.vue`
- `excel-ai-client/src/pages/workspaces/shared/SharedWorkspaceReport.vue`
- `excel-ai-client/src/services/workspaceApi.ts`

End-to-end flow: shared workspace dashboard

1. Public user opens a shared workspace token URL.
2. Frontend sends:

- `GET /api/share/workspace/{token}`

3. Backend route: `get_shared_workspace_dashboard()` in `app/routers/langfuse_share.py`.
4. The backend resolves the token back to a shared workspace and loads:

- the saved workspace dashboard shell
- the saved workspace widgets
- any persisted layout JSON

5. Typical response:

```
{
  "workspace_name": "Sales Analytics",
  "workspace_id": "workspace-id",
  "dashboard_id": "dashboard-id",
  "dashboard_name": "Sales Dashboard",
  "layout_json": {},
  "widgets": [],
  "source": "workspace"
}
```

6. The shared dashboard is rendered read-only.

End-to-end flow: shared workspace chat

1. Public user asks a question.
2. Frontend sends:

- `POST /api/share/workspace/{token}/chat`

3. Payload:

```
{
  "question": "Show top 5 regions by sales"
}
```

4. Backend route: `shared_workspace_chat()`.
5. It resolves the token, builds a `WorkspaceSqlAgent`, and runs the same SQL-answering path used by authenticated workspace chat.
6. Response mirrors normal workspace chat structure and is shown in the public chat UI.

End-to-end flow: shared workspace report

1. In AI mode, frontend sends:

- `POST /api/share/workspace/{token}/report/chat`

with:

```
{
  "question": "Show top 5 regions by sales",
  "title": "Top Regions"
}
```

2. In SQL mode, frontend sends:

- `POST /api/share/workspace/{token}/report/execute_custom_sql`

with:

```
{
  "title": "Custom SQL Report",
  "sql_query": "select region, sum(amount) as sales from fact_orders group by region"
}
```

3. For AI summary, frontend sends:

- `POST /api/share/workspace/{token}/report/summarize`

4. Payload example:

```
{
  "question": "Top Regions",
  "sql_query": "select region, sum(amount) as sales from fact_orders group by region",
  "columns": ["region", "sales"],
  "data": [
    {"region": "West", "sales": 1200}
  ]
}
```

5. Backend route: `shared_workspace_report_summarize()`.

6. The backend computes a compact statistical profile from the returned data and asks the LLM for a markdown summary.

7. Response:

```
{
  "status": "success",
  "summary": "West leads sales, followed by..."
}
```

End-to-end flow: open a shared report snapshot

1. A public user opens the report share URL.

2. Frontend loads:

- `GET /api/share/report/{token}`

3. Backend route: `get_shared_report()` in `langfuse_share.py`.

4. Response includes the stored report title, original question, SQL, columns, data, row count, AI summary, and workspace name.

5. The frontend renders the saved report exactly as it was shared.

Small But Important Support Flows

Langfuse trace/support flow

Frontend may call:

- `GET /api/langfuse-token`

with query params like:

```
{
  "session_id": "session-id",
  "trace_id": "trace-id",
  "target": "dashboard"
}
```

Backend route: `get_langfuse_token()` in `langfuse_share.py`.

This supports trace/session deep links used across chat and dashboard views.

What happens inside this route:

- the backend checks whether Langfuse session cookies already exist for the logged-in user
- it restores cookies from Redis when possible
- it refreshes the Langfuse session if needed
- it mints a short-lived one-time SSO key
- it returns an SSO bootstrap URL plus the target Langfuse URL

Typical response:

```
{
  "sso_url": "http://host/api/langfuse-sso?key=uuid",
  "langfuse_url": "http://langfuse-host/project/sessions/session-id",
  "session_id": "session-id"
}
```

Langfuse SSO helper flow

- `GET /api/langfuse-sso`

This helper route in `langfuse_share.py` is used for Langfuse SSO redirection/bootstrap support.

The route consumes the one-time key, sets Langfuse session cookies in the browser response, and issues a `302` redirect to the real Langfuse target URL.

Session-close flow

Legacy chat/session cleanup uses:

- `POST /api/session/close`

handled by `close_session_beacon()` in `app/routers/chat.py`.

Payload:

```
{
  "file_uuid": "file-id",
  "session_id": "langfuse-session-id",
  "token": "jwt"
}
```

The backend validates the JWT, resolves the user, and inserts a synthetic leave event into `chat_history` unless that session has already been closed.

Typical responses:

```
{
  "status": "success"
}
```

or:

```
{
  "status": "already_closed"
}
```

Cache reset flow

If a file's semantic cache becomes stale, frontend calls:

- `DELETE /api/files/{file_identfier}/cache`

handled by `delete_file_cache()` in `files.py`.

The backend resolves the file identifier, enforces file access, clears Redis semantic cache entries for that file UUID, and returns:

```
{
  "status": "success",
  "file_uuid": "file-id"
}
```

Health/info flow

- `GET /health`
- `GET /api/info`

These come from `app/routers/health.py` and are mainly operational support endpoints.

The actual liveness route is `/health`, not `/api/health`.

`GET /health` typically returns:

```
{
  "status": "healthy",
  "database": "connected"
}
```

GET `/api/info` returns a lightweight API descriptor such as:

```
{
  "status": "ok",
  "api": "Excel Analysis API",
  "version": "1.0.0",
  "endpoints": {}
}
```

API Coverage Audit

This walkthrough has been cross-checked against every router registered in `excel-ai-server-api/main.py` and every route decorator under `excel-ai-server-api/app/routers/`.

Coverage summary:

- 15 router modules
- 143 HTTP endpoints
- 25 primary product flows plus support flows
- no currently registered API is left undocumented after this audit

Module-to-flow API matrix

Auth and admin

- Flow 1: POST `/api/auth/register`, POST `/api/auth/login`, GET `/api/auth/me`, POST `/api/auth/logout`, POST `/api/auth/change-password`
- Flow 1: GET `/api/admin/users`, POST `/api/admin/users`, PATCH `/api/admin/users/{email}`

File ingestion and file operations

- Flow 2: GET `/api/files/upload-progress/{file_uuid}`, POST `/api/files/upload`, GET `/api/files`, PATCH `/api/files/{file_uuid}/move`, PATCH `/api/files/{file_uuid}/metadata`, GET `/api/files/{file_identifier}`, GET `/api/files/{file_identifier}/chunks`, DELETE `/api/files/{file_identifier}`, POST `/api/files/{file_identifier}/restore`, DELETE `/api/files/{file_identifier}/permanent`, GET `/api/files/trash`, POST `/api/files/bulk-move`, DELETE `/api/files/trash/empty`
- Flow 3: POST `/api/files/upload-raw`, POST `/api/files/extract-sheet`, GET `/api/files/groups`
- Support flows: DELETE `/api/files/{file_identifier}/cache`

Preview, saved questions, query, and legacy chat

- Flow 4: GET `/api/preview/{file_identifier}`, GET `/api/statistics/{file_identifier}`, GET `/api/categories`, PUT `/api/categories/rename`, DELETE `/api/categories/{category_name}`, POST `/api/questions/save`, GET `/api/questions/list`, GET `/api/questions/list/{file_uuid}`, PUT `/api/questions/{question_id}`, DELETE `/api/questions/{question_id}`, POST `/api/query/batch`
- Flow 5: POST `/api/query`, GET `/api/chat/{file_uuid}`, POST `/api/chat/{file_uuid}`, DELETE `/api/chat/{file_uuid}`, POST `/api/chat/{file_uuid}/soft_delete`
- Support flows: POST `/api/session/close`

Legacy dashboard and export

- Flow 6: GET `/api/dashboard/{file_identifier}`, POST `/api/dashboard/{file_identifier}/generate`, POST `/api/dashboard/{file_identifier}/widget`, POST `/api/dashboard/{file_identifier}/update-widgets`, POST `/api/dashboard/{file_identifier}/clone-template-widgets`, POST `/api/export/{file_identifier}`
- Flow 7: POST `/api/dashboard/{file_identifier}/filter`, GET `/api/dashboard/{file_identifier}/schema`, POST `/api/dashboard/{file_identifier}/chart-builder`
- Flow 8: POST `/api/dashboard/clone-widgets`, POST `/api/dashboard/compare-unified`

Projects and boards

- Flow 9: GET `/api/projects`, POST `/api/projects`, POST `/api/projects/{project_id}/subprojects`, PUT `/api/projects/{project_id}`, PUT `/api/projects/{project_id}/subprojects/{subproject_id}`, DELETE `/api/projects/{project_id}`, DELETE `/api/projects/{project_id}/subprojects/{subproject_id}`, GET `/api/projects/{project_id}/dashboard`, PUT `/api/projects/{project_id}/active-file`, PUT `/api/projects/{project_id}/active-file-stream`, POST `/api/projects/{project_id}/dashboard/save`
- Flow 10: GET `/api/boards`, POST `/api/boards`, DELETE `/api/boards/{board_id}`, POST `/api/boards/{board_id}/upload`, GET `/api/boards/{board_id}/dashboard`, POST `/api/boards/{board_id}/dashboard/save`, PUT `/api/boards/{board_id}/active-file`, PUT

```
/api/boards/{board_id}/active-file-stream, POST /api/boards/{board_id}/share, PUT /api/boards/{board_id}/publish, PUT /api/boards/{board_id}/unpublish
```

Shared and public consumption

- Flow 11: GET /api/share/{token}, POST /api/share/{token}/filter
- Flow 25: GET /api/share/workspace/{token}, POST /api/share/workspace/{token}/chat, POST /api/share/workspace/{token}/report/chat, POST /api/share/workspace/{token}/report/execute_custom_sql, POST /api/share/workspace/{token}/report/summarize, GET /api/share/report/{token}
- Support flows: GET /api/langfuse-token, GET /api/langfuse-sso

ETL and workspace platform

- Flow 12: POST /api/workspaces, GET /api/workspaces/{workspace_id}/etl-connection
- Flow 13: GET /api/workspaces, PATCH /api/workspaces/{workspace_id}, DELETE /api/workspaces/{workspace_id}
- Flow 14: POST /api/etl/connect, POST /api/etl/preview-table, GET /api/etl/connections, DELETE /api/etl/connections/{connection_id}, GET /api/etl/datasets
- Flow 15: POST /api/etl/generate-transform, POST /api/etl/dry-run
- Flow 16: POST /api/etl/execute-job, GET /api/etl/job/{job_id}, GET /api/etl/connection/{connection_id}/jobs, GET /api/etl/job/{job_id}/detail, PUT /api/etl/job/{job_id}, PATCH /api/etl/job/{job_id}/sync-config
- Flow 17: POST /api/etl/job/{job_id}/sync, POST /api/etl/sync-update
- Flow 18: GET /api/workspaces/{workspace_id}, GET /api/workspaces/{workspace_id}/tables, GET /api/workspaces/{workspace_id}/tables/{table_name}/columns, PATCH /api/workspaces/{workspace_id}/tables/{table_name}/description, PATCH /api/workspaces/{workspace_id}/columns/{column_id}/description
- Flow 19: POST /api/workspaces/{workspace_id}/profile, POST /api/workspaces/{workspace_id}/profile/{table_name}
- Flow 20: GET /api/workspaces/{workspace_id}/semantic, GET /api/workspaces/{workspace_id}/relationships, POST /api/workspaces/{workspace_id}/relationships, DELETE /api/workspaces/{workspace_id}/relationships/{relationship_id}, POST /api/workspaces/{workspace_id}/semantic/metrics, DELETE /api/workspaces/{workspace_id}/semantic/metrics/{metric_id}, POST /api/workspaces/{workspace_id}/semantic/dimensions, DELETE /api/workspaces/{workspace_id}/semantic/dimensions/{dimension_id}, POST /api/workspaces/{workspace_id}/semantic/synonyms, DELETE /api/workspaces/{workspace_id}/semantic/synonyms/{synonym_id}
- Flow 21: GET /api/workspaces/{workspace_id}/chat/history, DELETE /api/workspaces/{workspace_id}/chat/history, DELETE /api/workspaces/{workspace_id}/chat/history/{message_id}, POST /api/workspaces/{workspace_id}/chat
- Flow 22: POST /api/workspaces/{workspace_id}/report/execute_custom_sql, POST /api/workspaces/{workspace_id}/report/summarize, POST /api/workspaces/{workspace_id}/report/share
- Flow 23: POST /api/workspaces/{workspace_id}/share/toggle, GET /api/workspaces/{workspace_id}/share/status, GET /api/workspaces/{workspace_id}/dashboard/stream, GET /api/workspaces/{workspace_id}/dashboard, POST /api/workspaces/{workspace_id}/dashboard/save
- Flow 24: POST /api/workspaces/{workspace_id}/dashboard/pin, DELETE /api/workspaces/{workspace_id}/dashboard/widgets/{widget_id}, POST /api/workspaces/{workspace_id}/dashboard/widgets/{widget_id}/refresh, POST /api/workspaces/{workspace_id}/dashboard/widget, POST /api/workspaces/{workspace_id}/dashboard/widget/custom, GET /api/workspaces/{workspace_id}/dashboard/schema, POST /api/workspaces/{workspace_id}/dashboard/chart-builder

Operational endpoints

- Support flows: GET /health, GET /api/info

API Contract Reference

This appendix is the compact per-endpoint contract layer for the walkthrough. Every implemented API is listed here with its flow context, input shape, backend handler, main connected modules, and response contract.

auth.py

Endpoint	Flow / Trigger	Input	Handler / Connected Modules	Response
POST /api/auth/register	Flow 1. Register	Body RegisterRequest { email, username, password, name?, role }	register() -> app/core/auth.py; JWT/user storage	AuthResponse or JSON matching that model
POST /api/auth/login	Flow 1. Login	Body LoginRequest { email?, loginId?, companyId?, password }	login() -> app/core/auth.py; JWT/user storage	AuthResponse or JSON matching that model
GET /api/auth/me	Flow 1. Get current user info	No body	get_current_user_info() -> app/core/auth.py; JWT/user storage	UserInfo or JSON matching that model
POST /api/auth/logout	Flow 1. Logout	No body	logout() -> app/core/auth.py; JWT/user storage	Status/result JSON
POST /api/auth/change-password	Flow 1. Change password	Body ChangePasswordRequest { current_password, new_password }	change_password() -> app/core/auth.py; JWT/user storage	Status/result JSON

admin.py

Endpoint	Flow / Trigger	Input	Handler / Connected Modules	Response
GET /api/admin/users	Flow 1. Admin list users	No body	admin_list_users() -> app/core/auth.py; JWT/user storage	List[AdminUserResponse] or JSON matching that model
POST /api/admin/users	Flow 1. Admin create user	Body AdminCreateUserRequest { email, username, password, name?, role }	admin_create_user() -> app/core/auth.py; JWT/user storage	AdminUserResponse or JSON matching that model
PATCH /api/admin/users/{email}	Flow 1. Admin update user	Path email; Body AdminUpdateUserRequest { is_active?, role?, name?, username?, password? }	admin_update_user() -> app/core/auth.py; JWT/user storage	AdminUserResponse or JSON matching that model

files.py

Endpoint	Flow / Trigger	Input	Handler / Connected Modules	Response
GET /api/files/upload-progress/{file_uuid}	Flow 2. Server-Sent Events endpoint for real-time upload progress. Frontend subscribes to this after starting an upload.	Path file_uuid	upload_progress_stream() -> app/services/progress_manager.py, app/core/agent.py, app/core/ingestion.py	SSE progress event: { stage, current total, message }
POST /api/files/upload	Flow 2. Upload an Excel or CSV file for analysis.	Multipart file, file_uuid, optional project_id, subproject_id	upload_excel_file() -> app/services/progress_manager.py, app/core/agent.py, app/core/ingestion.py	Upload result { status, message, file_uuid, filename, table_name, rows columns }
POST /api/files/upload-raw	Flow 3. Step 1 of multi-sheet upload: receive raw .xlsx/xls, return sheet list.	Multipart file upload	upload_raw_file() -> file registry, temp storage, DatabaseManager	UploadRawResponse or JSON matching that model
POST /api/files/extract-sheet	Flow 3. Step 2 of multi-sheet upload: extract one sheet from previously uploaded raw file.	Body ExtractSheetRequest { temp_id, sheet_name, existing_group_id?, project_id?, subproject_id? }	extract_sheet() -> file registry, temp storage, DatabaseManager	Extract result { status, file_uuid, group_id, sheet_name, filename, table_name, rows columns }
GET /api/files/groups	Flow 3. Return all file_groups belonging to the current user.	No body	list_file_groups() -> file registry, temp storage, DatabaseManager	Grouped workbook list { groups: [...] }
GET /api/files	Flow 2. List ACTIVE uploaded Excel files (Excludes deleted ones).	No body	list_files() -> file registry, temp storage, DatabaseManager	FileListResponse or JSON matching that model
GET /api/files/trash	Flow 2. List temporarily deleted files (Recycle Bin).	No body	list_trash_files() -> app/core/file_ops.py; file registry + table lifecycle	FileListResponse or JSON matching that model
PATCH /api/files/{file_uuid}/move	Flow 2. Move a file to a different project/subproject.	Path file_uuid; Body FileMoveRequest { target_folder_id?, target_subfolder_id? }	move_file() -> file registry, temp storage, DatabaseManager	Status/result JSON
PATCH /api/files/{file_uuid}/metadata	Flow 2. Update file metadata (pinned, favorite, tags).	Path file_uuid; Body FileMetadataUpdateRequest { is_pinned?, is_favorite?, tags? }	update_file_metadata() -> file registry, temp storage, DatabaseManager	Status/result JSON

Endpoint	Flow / Trigger	Input	Handler / Connected Modules	Response
POST /api/files/bulk-move	Flow 2. Move multiple files to a different project/subproject.	Body <code>BulkMoveRequest</code> { <code>file_ids</code> , <code>target_folder_id?</code> , <code>target_subfolder_id?</code> }	<code>bulk_move_files()</code> -> file registry, temp storage, <code>DatabaseManager</code>	Status JSON with moved-file summary
GET /api/files/{file_identifier}	Flow 2. Get details about a single file.	Path <code>file_identifier</code>	<code>get_file_info()</code> -> file registry, temp storage, <code>DatabaseManager</code>	File metadata JSON for GET or status/message JSON for DELETE
GET /api/files/{file_identifier}/chunks	Flow 2. Get paginated data chunks from a file.	Path <code>file_identifier</code>	<code>get_file_chunks()</code> -> file registry, temp storage, <code>DatabaseManager</code>	Status/result JSON
DELETE /api/files/{file_identifier}	Flow 2. Soft-delete a file (move to trash).	Path <code>file_identifier</code>	<code>delete_file()</code> -> <code>app/core/file_ops.py</code> ; file registry + table lifecycle	File metadata JSON for GET or status/message JSON for DELETE
POST /api/files/{file_identifier}/restore	Flow 2. Restore a deleted file from trash.	Path <code>file_identifier</code>	<code>restore_file()</code> -> <code>app/core/file_ops.py</code> ; file registry + table lifecycle	Status/message JSON
DELETE /api/files/{file_identifier}/permanent	Flow 2. Permanently delete a file and its associated tables.	Path <code>file_identifier</code>	<code>permanent_delete_file()</code> -> <code>app/core/file_ops.py</code> ; file registry + table lifecycle	Status/message JSON
DELETE /api/files/trash/empty	Flow 2. Permanently delete all files in user's trash.	No body	<code>empty_trash()</code> -> <code>app/core/file_ops.py</code> ; file registry + table lifecycle	Status JSON with deleted count
DELETE /api/files/{file_identifier}/cache	Support. Clear semantic cache for a file.	Path <code>file_identifier</code>	<code>delete_file_cache()</code> -> <code>app/core/cache.py</code> ; file resolution/access checks	Status JSON with cleared <code>file_uuid</code>

preview.py

Endpoint	Flow / Trigger	Input	Handler / Connected Modules	Response
GET /api/preview/{file_identifier}	Flow 4. Preview file	Path <code>file_identifier</code> ; Query <code>rows</code>	<code>preview_file()</code> -> <code>app/dependencies.py</code> ; loaded file agent/dataframe stats	Status/result JSON
GET /api/statistics/{file_identifier}	Flow 4. Get statistics	Path <code>file_identifier</code>	<code>get_statistics()</code> -> <code>app/dependencies.py</code> ; loaded file agent/dataframe stats	Status/result JSON

categories.py

Endpoint	Flow / Trigger	Input	Handler / Connected Modules	Response
GET /api/categories	Flow 4. List all categories derived from <code>saved_questions.question_category</code> plus default categories (Generic, Specific) which are always shown.	No body	<code>list_categories()</code> -> <code>saved_questions</code> ; file-scope resolution	<code>List[CategoryItem]</code> matching that model
PUT /api/categories/rename	Flow 4. Rename a category by updating all <code>saved_questions</code> with <code>old_name</code> to <code>new_name</code> .	Body <code>CategoryRenameRequest</code> { <code>old_name</code> , <code>new_name</code> }	<code>rename_category()</code> -> <code>saved_questions</code> ; file-scope resolution	Status/result JSON
DELETE /api/categories/{category_name}	Flow 4. Delete a category by resetting all its questions to 'Uncategorized'.	Path <code>category_name</code>	<code>delete_category()</code> -> <code>saved_questions</code> ; file-scope resolution	Status/result JSON
POST /api/questions/save	Flow 4. Save/bookmark a question from a chat session. Automatically determines scope (ROOT/PROJECT/SUBPROJECT) from the file's location.	Body <code>SaveQuestionRequest</code> { <code>question_text</code> , <code>category</code> , <code>file_uuid</code> }	<code>save_question()</code> -> <code>saved_questions</code> ; file-scope resolution	<code>SavedQuestionResponse</code> JSON matching that model
GET /api/questions/list	Flow 4. Fetch ALL saved questions for the current user across all scopes. Questions are globally	No body	<code>list_saved_questions()</code> -> <code>saved_questions</code> ; file-scope resolution	<code>List[SavedQuestion]</code> or JSON matching that model

Endpoint	Flow / Trigger	Input	Handler / Connected Modules	Response
	accessible regardless of which project they were saved from. Deduplicates by question_text (keeps the most recently created).			
GET <code>/api/questions/list/{file_uuid}</code>	Flow 4. Fetch saved questions applicable to a target file scope. Includes ROOT + PROJECT + SUBPROJECT scopes relevant to the file.	Path <code>file_uuid</code>	<code>list_saved_questions_for_file()</code> -> <code>saved_questions</code> ; file-scope resolution	<code>List[SavedQuest]</code> or JSON matching
PUT <code>/api/questions/{question_id}</code>	Flow 4. Update a saved question's text and/or category (only if owned by caller).	Path <code>question_id</code> ; Body <code>body: dict</code>	<code>update_saved_question()</code> -> <code>saved_questions</code> ; file-scope resolution	Status/result JSON
DELETE <code>/api/questions/{question_id}</code>	Flow 4. Delete a saved question by ID (only if owned by caller).	Path <code>question_id</code>	<code>delete_saved_question()</code> -> <code>saved_questions</code> ; file-scope resolution	Status/result JSON

query.py

Endpoint	Flow / Trigger	Input	Handler / Connected Modules	Response
POST <code>/api/query</code>	Flow 5. Analyze query	Body <code>QueryRequest</code> { <code>file_uuid?</code> , <code>filename?</code> , <code>query</code> , <code>use_cache?</code> , <code>session_id?</code> , <code>source_type?</code> }	<code>analyze_query()</code> -> <code>app/core/routing.py</code> , <code>app/core/agent.py</code> , <code>app/core/paths/*</code>	<code>QueryResponse</code> or JSON matching that model
POST <code>/api/query/batch</code>	Flow 4. Batch execute queries	Body <code>BatchQueryRequest</code> { <code>file_uuid</code> , <code>questions</code> , <code>session_id?</code> }	<code>batch_execute_queries()</code> -> <code>app/core/routing.py</code> , <code>app/core/agent.py</code> , <code>app/core/paths/*</code>	<code>BatchQueryResponse</code> or JSON matching that model

chat.py

Endpoint	Flow / Trigger	Input	Handler / Connected Modules	Response
GET <code>/api/chat/{file_uuid}</code>	Flow 5. Get chat history	Path <code>file_uuid</code> ; Query <code>limit</code> , <code>offset</code>	<code>get_chat_history()</code> -> <code>chat_history</code> ; auth validation for beacon close	<code>List[ChatMessageResponse]</code> or JSON matching that model
POST <code>/api/chat/{file_uuid}</code>	Flow 5. Save chat message	Path <code>file_uuid</code> ; Body <code>ChatMessageSave</code> { <code>role</code> , <code>content</code> , <code>query_type?</code> , <code>cache_hit</code> , <code>response_time?</code> , <code>metadata?</code> }	<code>save_chat_message()</code> -> <code>chat_history</code> ; auth validation for beacon close	<code>ChatMessageResponse</code> or JSON matching that model
DELETE <code>/api/chat/{file_uuid}</code>	Flow 5. Clear chat history	Path <code>file_uuid</code>	<code>clear_chat_history()</code> -> <code>chat_history</code> ; auth validation for beacon close	Status/result JSON
POST <code>/api/session/close</code>	Support. Close session beacon	Body <code>SessionCloseRequest</code> { <code>file_uuid</code> , <code>session_id</code> , <code>token</code> }	<code>close_session_beacon()</code> -> <code>chat_history</code> ; auth validation for beacon close	Status JSON (<code>success</code> , <code>already_closed</code> , or <code>ignored</code>)
POST <code>/api/chat/{file_uuid}/soft_delete</code>	Flow 5. Soft delete chat message	Path <code>file_uuid</code> ; Body <code>ChatMessageSoftDelete</code> { <code>message_id</code> }	<code>soft_delete_chat_message()</code> -> <code>chat_history</code> ; auth validation for beacon close	Status/result JSON

dashboard.py

Endpoint	Flow / Trigger	Input	Handler / Connected Modules	Response
POST <code>/api/dashboard/{file_identifienr}/generate</code>	Flow 6. Generate (or regenerate) a dashboard with optional user-defined requirements.	Path <code>file_identifienr</code> ; Body <code>DashboardGenerateRequest</code> { <code>user_requirements?</code> , <code>session_id?</code> , <code>source_type?</code> , <code>mode?</code> }	<code>generate_dashboard_custom()</code> -> <code>app/core/agent.py</code> , <code>app/core/dashboard/kpi_builder.py</code> , formatter helpers	Status/result JSON

Endpoint	Flow / Trigger	Input	Handler / Connected Modules	Response
GET /api/dashboard/{file_identifier}	Flow 6. Get AI-powered KPI dashboard for a file.	Path <code>file_identifier</code> ; Query <code>regenerate</code> , <code>session_id</code> , <code>source_type</code> , <code>mode</code>	<code>generate_dashboard()</code> -> <code>app/core/agent.py</code> , <code>app/core/dashboard/kpi_builder.py</code> , formatter helpers	Status/result JSON
POST /api/dashboard/{file_identifier}/widget	Flow 6. Generate a single dashboard widget from natural language query.	Path <code>file_identifier</code> ; Body <code>DashboardWidgetRequest</code> { <code>query</code> , <code>compare_file_id?</code> , <code>session_id?</code> , <code>source_type?</code> , <code>widget_type_hint?</code> , <code>chart_type_hint?</code> , <code>filter_context_hint?</code> , <code>mode?</code> }	<code>generate_dashboard_widget()</code> -> <code>app/core/agent.py</code> , <code>app/core/dashboard/kpi_builder.py</code> , formatter helpers	Status/result JSON
POST /api/dashboard/clone-widgets	Flow 8. Clone dashboard widgets from base file to target file for comparison.	Body <code>CloneWidgetsRequest</code> { <code>base_file_id</code> , <code>target_file_id</code> , <code>session_id?</code> }	<code>clone_widgets_for_compare()</code> -> <code>app/core/agent.py</code> , <code>app/core/dashboard/kpi_builder.py</code> , formatter helpers	Status/result JSON
POST /api/dashboard/compare-unified	Flow 8. Unified semantic comparison: merge data from both files into single widgets.	Body <code>UnifiedCompareRequest</code> { <code>base_file_id</code> , <code>compare_file_id</code> , <code>session_id?</code> }	<code>compare_unified()</code> -> <code>app/core/agent.py</code> , <code>app/core/dashboard/kpi_builder.py</code> , formatter helpers	Status/result JSON
POST /api/dashboard/{file_identifier}/filter	Flow 7. Cross-filter the dashboard using filtered data.	Path <code>file_identifier</code> ; Body <code>DashboardFilterRequest</code> { <code>column?</code> , <code>value?</code> , <code>filters?</code> , <code>session_id?</code> , <code>source_type?</code> }	<code>filter_dashboard()</code> -> <code>app/core/agent.py</code> , <code>app/core/dashboard/kpi_builder.py</code> , formatter helpers	Status/result JSON
POST /api/dashboard/{file_identifier}/update-widgets	Flow 6. Update dashboard widgets after user adds/removes widgets.	Path <code>file_identifier</code>	<code>update_dashboard_widgets()</code> -> <code>app/core/agent.py</code> , <code>app/core/dashboard/kpi_builder.py</code> , formatter helpers	Status/result JSON
GET /api/dashboard/{file_identifier}/schema	Flow 7. Return a lightweight schema of the dataset for the Chart Builder UI.	Path <code>file_identifier</code> ; Query <code>source_type</code> , <code>mode</code>	<code>get_dashboard_schema()</code> -> <code>app/core/agent.py</code> , <code>app/core/dashboard/kpi_builder.py</code> , formatter helpers	Status/result JSON
POST /api/dashboard/{file_identifier}/clone-template-widgets	Flow 6. Clone explicitly provided payload widgets (Chart Builder style).	Path <code>file_identifier</code> ; Body <code>TemplateCloneRequest</code> { <code>template_widgets</code> , <code>session_id?</code> }	<code>clone_template_widgets()</code> -> <code>app/core/agent.py</code> , <code>app/core/dashboard/kpi_builder.py</code> , formatter helpers	Status/result JSON
POST /api/dashboard/{file_identifier}/chart-builder	Flow 7. Generate a single chart widget interactively - no LLM, instant Pandas result.	Path <code>file_identifier</code> ; Body <code>ChartBuilderRequest</code> { <code>chart_type</code> , <code>dimension</code> , <code>measure?</code> , <code>aggregation</code> , <code>source_type?</code> , <code>mode?</code> }	<code>generate_custom_chart()</code> -> <code>app/core/agent.py</code> , <code>app/core/dashboard/kpi_builder.py</code> , formatter helpers	Status/result JSON

export.py

Endpoint	Flow / Trigger	Input	Handler / Connected Modules	Response
POST <code>/api/export/{file_identifiler}</code>	Flow 6. Export file	Path <code>file_identifiler</code> ; Query <code>format</code> ; Body <code>format: str</code>	<code>export_file()</code> -> <code>app/dependencies.py</code> , loaded dataframe agent, pandas serialization	Export JSON with <code>format</code> and serialized data

projects.py

Endpoint	Flow / Trigger	Input	Handler / Connected Modules
GET <code>/api/projects</code>	Flow 9. List projects with their subprojects for the current user.	No body	<code>list_projects()</code> -> <code>projects</code> , <code>subprojects</code> ; dashboard loaders file switching helpers
POST <code>/api/projects</code>	Flow 9. Create project	Body <code>ProjectCreateRequest</code> { <code>name</code> , <code>color?</code> , <code>is_dashboard</code> , <code>source_file_uuid?</code> }	<code>create_project()</code> -> <code>projects</code> , <code>subprojects</code> ; dashboard loaders file switching helpers
POST <code>/api/projects/{project_id}/subprojects</code>	Flow 9. Create subproject	Path <code>project_id</code> ; Body <code>SubprojectCreateRequest</code> { <code>name</code> }	<code>create_subproject()</code> -> <code>projects</code> , <code>subprojects</code> ; dashboard loaders file switching helpers
POST <code>/api/projects/{project_id}/dashboard/save</code>	Flow 9. Save widget positions/layout for the project dashboard (dashboard-first persistence).	Path <code>project_id</code>	<code>save_project_dashboard_layout()</code> -> <code>projects</code> / <code>subprojects</code> ; dashboard loaders; active-file switching helpers
PUT <code>/api/projects/{project_id}/active-file</code>	Flow 9. Switch the active data source for a dashboard project. Dashboard-first: loads the saved project dashboard template and clones it onto the new file's data via <code>clone_widgets_from_blueprints</code> .	Path <code>project_id</code> ; Body <code>ActiveFileRequest</code> { <code>file_uuid</code> , <code>session_id?</code> }	<code>update_active_file()</code> -> <code>projects</code> , <code>subprojects</code> ; dashboard loaders file switching helpers
GET <code>/api/projects/{project_id}/dashboard</code>	Flow 9. Canonical project-centric dashboard load. Dashboard-first approach: one dashboard per project stored in <code>dashboards</code> table. When the active file differs from what was last rendered, the saved widget template is cloned onto the new file's data via <code>clone_widgets_from_blueprints</code> .	Path <code>project_id</code>	<code>get_project_dashboard()</code> -> <code>projects</code> / <code>subprojects</code> ; dashboard loaders file switching helpers
PUT <code>/api/projects/{project_id}/active-file-stream</code>	Flow 9. Stream widgets one-by-one as NDJSON for project dashboards.	Path <code>project_id</code> ; Body <code>ActiveFileRequest</code> { <code>file_uuid</code> , <code>session_id?</code> }	<code>update_project_active_file_stream()</code> -> <code>projects</code> / <code>subprojects</code> ; dashboard loaders; active-file switching helpers
DELETE <code>/api/projects/{project_id}/subprojects/{subproject_id}</code>	Flow 9. Delete a subproject and all its files (CASCADE DELETE).	Path <code>project_id</code> , <code>subproject_id</code>	<code>delete_subproject()</code> -> <code>projects</code> , <code>subprojects</code> ; dashboard loaders file switching helpers
PUT <code>/api/projects/{project_id}</code>	Flow 9. Rename or update color of a folder (project).	Path <code>project_id</code> ; Body <code>ProjectUpdateRequest</code> { <code>name?</code> , <code>color?</code> }	<code>update_project()</code> -> <code>projects</code> , <code>subprojects</code> ; dashboard loaders file switching helpers
PUT <code>/api/projects/{project_id}/subprojects/{subproject_id}</code>	Flow 9. Rename a subfolder (subproject).	Path <code>project_id</code> , <code>subproject_id</code> ; Body <code>SubprojectUpdateRequest</code> { <code>name?</code> }	<code>update_subproject()</code> -> <code>projects</code> , <code>subprojects</code> ; dashboard loaders file switching helpers
DELETE <code>/api/projects/{project_id}</code>	Flow 9. Delete a project, all its subprojects, and all files (CASCADE DELETE).	Path <code>project_id</code>	<code>delete_project()</code> -> <code>projects</code> , <code>subprojects</code> ; dashboard loaders file switching helpers

boards.py

Endpoint	Flow / Trigger	Input	Handler / Connected Modules	Response
GET <code>/api/boards</code>	Flow 10. List all Insight	No body	<code>list_boards()</code> -> <code>insight_boards</code> , <code>board_files</code> ; board agent + studio-	<code>BoardListResponse</code> or JSON matching that

Endpoint	Flow / Trigger	Input	Handler / Connected Modules	Response
	Boards for the current user.		state helpers	model
POST /api/boards	Flow 10. Create a new empty Insight Board.	Body <code>BoardCreateRequest { name }</code>	<code>create_board()</code> -> <code>insight_boards</code> , <code>board_files</code> ; board agent + studio-state helpers	<code>BoardInfo</code> or JSON matching that model
DELETE /api/boards/{board_id}	Flow 10. Delete an Insight Board and its files (CASCADE).	Path <code>board_id</code>	<code>delete_board()</code> -> <code>insight_boards</code> , <code>board_files</code> ; board agent + studio-state helpers	Status/result JSON
POST /api/boards/{board_id}/upload	Flow 10. Lightweight file upload for Insight Boards - DataFrame only, no embeddings.	Path <code>board_id</code> ; Multipart file upload	<code>upload_board_file()</code> -> <code>insight_boards</code> , <code>board_files</code> ; board agent + studio-state helpers	Compatibility JSON with <code>compatible</code> , <code>missing_columns</code> , <code>extra_columns</code>
GET /api/boards/{board_id}/dashboard	Flow 10. Load an Insight Board's dashboard.	Path <code>board_id</code>	<code>get_board_dashboard()</code> -> <code>insight_boards</code> , <code>board_files</code> ; board agent + studio-state helpers	Board dashboard payload { <code>project</code> , <code>files</code> , <code>dashboard_data</code> }
POST /api/boards/{board_id}/dashboard/save	Flow 10. Save Insight Board widget layout.	Path <code>board_id</code> ; Body <code>BoardDashboardSaveRequest { widgets, screens, active_screen_id?, active_theme?, thumbnail_version?, screen_widgets, file_screen_widgets, file_screen_needs_generation, ... }</code>	<code>save_board_dashboard()</code> -> <code>insight_boards</code> , <code>board_files</code> ; board agent + studio-state helpers	Status/message JSON
PUT /api/boards/{board_id}/active-file	Flow 10. Switch the active data source for an Insight Board.	Path <code>board_id</code> ; Body <code>ActiveFileRequest { file_uuid, session_id? }</code>	<code>update_board_active_file()</code> -> <code>insight_boards</code> , <code>board_files</code> ; board agent + studio-state helpers	Status JSON with <code>active_file_uuid</code> and <code>dashboard_data</code>
PUT /api/boards/{board_id}/active-file-stream	Flow 10. Stream widgets one-by-one as NDJSON for side-by-side loading.	Path <code>board_id</code> ; Body <code>ActiveFileRequest { file_uuid, session_id? }</code>	<code>update_board_active_file_stream()</code> -> <code>insight_boards</code> , <code>board_files</code> ; board agent + studio-state helpers	NDJSON widget stream; final completion event
POST /api/boards/{board_id}/share	Flow 10. Toggle public sharing for an Insight Board.	Path <code>board_id</code> ; Body <code>BoardPublishRequest { widgets?, active_screen_id? }</code>	<code>toggle_board_share()</code> -> <code>insight_boards</code> , <code>board_files</code> ; board agent + studio-state helpers	Status JSON with share token(s) and published file info
PUT /api/boards/{board_id}/publish	Flow 10. Freeze current board state as published snapshot for	Path <code>board_id</code> ; Body <code>BoardPublishRequest { widgets?, active_screen_id? }</code>	<code>publish_board()</code> -> <code>insight_boards</code> , <code>board_files</code> ; board agent + studio-state helpers	Status/message JSON with <code>published_file_uuid</code>

Endpoint	Flow / Trigger	Input	Handler / Connected Modules	Response
	shared viewers.			
PUT /api/boards/{board_id}/unpublish	Flow 10. Clear published snapshot so shared viewers see the live dashboard state.	Path board_id	unpublish_board() -> insight_boards, board_files; board agent + studio-state helpers	Status/message JSON

health.py

Endpoint	Flow / Trigger	Input	Handler / Connected Modules	Response
GET /health	Support. Health check endpoint.	No body	health_check() -> app/core/database.py	Health JSON { status, database }
GET /api/info	Support. API information and available endpoints.	No body	api_info() -> app/core/database.py	API metadata/version JSON

langfuse_share.py

Endpoint	Flow / Trigger	Input	Handler / Connected Modules
GET /api/langfuse-token		Support. Step 1 of per-user SSO (requires Vue JWT). Looks up cached Langfuse session cookies for the logged-in user, mints a short-lived one-time key tied to that user's email, and returns a sso_url pointing at /api/langfuse-sso?key=<key> on localhost:8000. Vue opens that URL in a new tab. If the background task fired at login has not finished yet, this returns 503 so the Vue "Refresh session" button can retry shortly.	get_langfuse_token() -> app/services/langfuse_sso.py; Redirected cookie/session bridge
GET /api/langfuse-sso		Support. Step 2 of SSO with no JWT required because the one-time key is the credential. Validates the key, consumes it immediately, sets Langfuse NextAuth session cookies in the response, and issues a 302	langfuse_sso_redirect() -> app/services/langfuse_sso.py; Redirected cookie/session bridge

Endpoint	Flow / Trigger	Input	Handler / Connected Modules
	redirect to <code>localhost:3000</code> so the browser lands in Langfuse already authenticated.		
<code>GET /api/share/{token}</code>	Flow 11. Public read-only endpoint for shared dashboards from projects and boards.	Path <code>token</code>	<code>get_shared_dashboard()</code> -> <code>projects.py</code> , <code>boards.py</code> , <code>get_agent()</code> public share-token resolution
<code>POST /api/share/{token}/filter</code>	Flow 11. Public cross-filter endpoint for shared dashboards. No authentication required because the share token acts as the access key. Accepts single <code>{column, value}</code> or multi <code>{filters: {col: val, ...}}</code> .	Path <code>token</code> ; Body <code>SharedDashboardFilterRequest</code> <code>{ column?, value?, filters?, session_id? }</code>	<code>filter_shared_dashboard()</code> -> <code>projects.py</code> , <code>boards.py</code> , <code>get_agent()</code> public share-token resolution
<code>GET /api/share/workspace/{token}</code>	Flow 25. Public read-only endpoint for shared workspace dashboards. No auth required.	Path <code>token</code>	<code>get_shared_workspace_dashboard()</code> <code>app/core/agents/workspace_sql.py</code> shared workspace loaders
<code>POST /api/share/workspace/{token}/chat</code>	Flow 25. Public chat endpoint for shared workspace chatbots. No auth required. Resolves the share token to a workspace and runs the RAG-to- SQL pipeline.	Path <code>token</code> ; Body <code>_SharedWorkspaceChatRequest { question }</code>	<code>shared_workspace_chat()</code> -> <code>app/core/agents/workspace_sql.py</code> shared workspace loaders
<code>POST /api/share/workspace/{token}/report/chat</code>	Flow 25. Public AI-driven report endpoint for shared workspaces. No auth required.	Path <code>token</code> ; Body <code>_SharedReportChatRequest { question, title }</code>	<code>shared_workspace_report_chat()</code> - <code>app/core/agents/workspace_sql.py</code> shared workspace loaders
<code>POST /api/share/workspace/{token}/report/execute_custom_sql</code>	Flow 25. Public custom SQL report endpoint for shared workspaces. <code>SELECT</code> -only and no auth required.	Path <code>token</code> ; Body <code>_SharedReportSqlRequest { title, sql_query }</code>	<code>shared_workspace_report_sql()</code> -> <code>app/core/agents/workspace_sql.py</code> shared workspace loaders
<code>POST /api/share/workspace/{token}/report/summarize</code>	Flow 25. Public AI summarization for shared workspace reports. No auth required.	Path <code>token</code> ; Body <code>_SharedReportSummarizeRequest</code> <code>{ question, sql_query, columns, data }</code>	<code>shared_workspace_report_summarize()</code> -> <code>app/core/agents/workspace_sql.py</code> shared workspace loaders

Endpoint	Flow / Trigger	Input	Handler / Connected Modules
GET /api/share/report/{token}	Flow 25. Public endpoint to view a shared report snapshot. No auth required.	Path token	get_shared_report() -> projects.boards.py, get_agent(), public share token resolution

etl.py

Endpoint	Flow / Trigger	Input	Handler / Connected Modules	Response
POST /api/etl/connect	Flow 14. Connect to an external database and return available tables or collections. The connection is saved for subsequent preview and extract operations.	Body <code>ETLConnectRequest { connection_id?, db_type?, host, port, username, password, database, auth_source, ... }</code>	<code>etl_connect()</code> -> <code>app/core/etl/connectors/*</code> , connector factory	ETLConnect matching
POST /api/etl/preview-table	Flow 14. Fetch sample rows from an external database table for preview.	Body <code>ETLPreviewRequest { connection_id, table_name, limit }</code>	<code>etl_preview_table()</code> -> <code>app/core/etl/connectors/*</code> , connector factory	ETLPreview matching
POST /api/etl/dry-run	Flow 15. Run the full ETL pipeline on only 100 rows for quick preview. Extract -> transform in sandbox -> return output preview.	Body <code>ETLDryRunRequest { connection_id, table_names, datasets, transform_script }</code>	<code>etl_dry_run()</code> -> <code>app/core/etl/sandbox/executor.py</code> ; transform preview path	ETLDryRun matching
POST /api/etl/execute-job	Flow 16. Submit a full ETL pipeline job. Runs in the background: extract full data -> transform in sandbox -> load into PostgreSQL.	Body <code>ETLExecuteRequest { job_id?, connection_id, table_names, datasets, transform_script, target_table?, pipeline_name?, sync_mode, ... }</code>	<code>etl_execute_job()</code> -> <code>app/core/etl/loader.py</code> , connectors, ETL job metadata tables	ETLJobRequest matching
GET /api/etl/job/{job_id}	Flow 16. Poll the status of an ETL job.	Path job_id	<code>etl_job_status()</code> -> <code>app/core/etl/loader.py</code> , connectors, ETL job metadata tables	ETLJobStatus matching
GET /api/etl/connection/{connection_id}/jobs	Flow 16. Get all previously created pipelines or jobs for a specific connection.	Path connection_id	<code>etl_get_connection_jobs()</code> -> <code>app/core/etl/loader.py</code> , connectors, ETL job metadata tables	Status/res
POST /api/etl/job/{job_id}/sync	Flow 17. Incremental or delta pull. Fetches new source rows using the high-water mark and replaces <code>{{LAST_SYNC_VALUE}}</code> in the original query with the actual MAX value from the target table.	Path job_id; Body <code>ETLDeltaSyncRequest { sync_column?, preview_only, primary_keys? }</code>	<code>etl_delta_sync()</code> -> <code>app/core/etl/loader.py</code> , connectors, ETL job metadata tables	ETLSyncRequest JSON mat
GET /api/etl/connections	Flow 14. List all saved ETL connections for the current user.	No body	<code>etl_list_connections()</code> -> <code>app/core/etl/loader.py</code> , connectors, ETL job metadata tables	ETLConnections JSON mat
DELETE /api/etl/connections/{connection_id}	Flow 14. Remove a saved ETL connection.	Path connection_id	<code>etl_delete_connection()</code> -> <code>app/core/etl/loader.py</code> , connectors, ETL job metadata tables	Status/res
GET /api/etl/datasets	Flow 14. List all ETL datasets grouped by connection and job.	No body	<code>etl_list_datasets()</code> -> <code>app/core/etl/loader.py</code> , connectors, ETL job metadata tables	ETLDatasets JSON mat

Endpoint	Flow / Trigger	Input	Handler / Connected Modules	Response
POST /api/etl/sync-update	Flow 17. Sync changes from the source DB into an existing destination table using append mode.	Body <code>ETLSyncUpdateRequest</code> { connection_id, workspace_id, target_table_name, table_names, datasets, transform_script? }	<code>etl_sync_update()</code> -> <code>app/core/etl/loader.py</code> , connectors, ETL job metadata tables	ETLSyncL JSON ma
POST /api/etl/generate-transform	Flow 15. Generate a transform script from a prompt plus source-table context.	Body <code>ETLGenerateTransformRequest</code> { prompt, tables }	<code>generate_transform()</code> -> LLM transform generation + ETL schemas	ETLGener or JSON r
PATCH /api/etl/job/{job_id}/sync-config	Flow 16. Update <code>sync_column</code> and or <code>sync_mode_type</code> on an existing pipeline job.	Path <code>job_id</code> ; Body <code>_SyncConfigRequest</code> { sync_column?, sync_mode_type?, pipeline_name? }	<code>etl_update_sync_config()</code> -> <code>app/core/etl/loader.py</code> , connectors, ETL job metadata tables	Status/res
GET /api/etl/job/{job_id}/detail	Flow 16. Return all configuration fields for a pipeline job so the edit wizard can be pre-filled.	Path <code>job_id</code>	<code>etl_get_job_detail()</code> -> <code>app/core/etl/loader.py</code> , connectors, ETL job metadata tables	Status/res
PUT /api/etl/job/{job_id}	Flow 16. Update an existing pipeline job configuration in place without re-running it.	Path <code>job_id</code> ; Body <code>_UpdateJobRequest</code> { source_tables?, transform_script?, target_table?, sync_mode?, sync_column?, sync_mode_type?, pipeline_name?, primary_keys? }	<code>etl_update_job()</code> -> <code>app/core/etl/loader.py</code> , connectors, ETL job metadata tables	Status/res

workspace.py

Endpoint	Flow / Trigger	Input	Handler / Coni
POST /api/workspaces	Flow 12. Create a new workspace with its own PostgreSQL schema.	Body <code>WorkspaceCreateRequest</code> { name, description? }	<code>create_workspa</code> <code>DatabaseManag</code> + registry table
GET /api/workspaces	Flow 13. List all workspaces for the current user.	No body	<code>list_workspa</code> <code>DatabaseManag</code> + registry table
GET /api/workspaces/{workspace_id}	Flow 18. Get workspace detail with all tables and their column metadata.	Path <code>workspace_id</code>	<code>get_workspa</code> <code>DatabaseManag</code> + registry table
PATCH /api/workspaces/{workspace_id}	Flow 13. Update workspace name and description attributes.	Path <code>workspace_id</code> ; Body <code>WorkspaceUpdateRequest</code> { name?, description? }	<code>update_workspa</code> <code>DatabaseManag</code> + registry table
DELETE /api/workspaces/{workspace_id}	Flow 13. Delete a workspace and its PostgreSQL schema.	Path <code>workspace_id</code>	<code>delete_workspa</code> <code>DatabaseManag</code> + registry table
POST /api/workspaces/{workspace_id}/share/toggle	Flow 23. Toggle sharing on or off for a workspace and generate a <code>share_token</code> if needed.	Path <code>workspace_id</code>	<code>toggle_workspa</code> <code>DatabaseManag</code> + registry table
GET /api/workspaces/{workspace_id}/share/status	Flow 23. Get current sharing status and token for a workspace.	Path <code>workspace_id</code>	<code>get_workspa</code> <code>DatabaseManag</code> + registry table
GET /api/workspaces/{workspace_id}/tables	Flow 18. List all tables in a workspace with basic metadata.	Path <code>workspace_id</code>	<code>list_workspa</code> <code>DatabaseManag</code> metadata + ser

Endpoint	Flow / Trigger	Input	Handler / Coni
PATCH /api/workspaces/{workspace_id}/tables/{table_name}/description	Flow 18. Update a table's business description as user-editable semantic context.	Path workspace_id, table_name; Body UpdateTableDescriptionRequest { description }	update_table_ DatabaseManag metadata + ser
GET /api/workspaces/{workspace_id}/tables/{table_name}/columns	Flow 18. List columns for a specific table with metadata.	Path workspace_id, table_name	list_table_co DatabaseManag metadata + ser
PATCH /api/workspaces/{workspace_id}/columns/{column_id}/description	Flow 18. Update a column's business description.	Path workspace_id, column_id; Body UpdateColumnDescriptionRequest { description }	update_colum DatabaseManag metadata + ser
GET /api/workspaces/{workspace_id}/semantic	Flow 20. Get the full semantic layer with metrics, dimensions, and synonyms for a workspace.	Path workspace_id	get_semantic_ DatabaseManag metadata + ser
POST /api/workspaces/{workspace_id}/semantic/metrics	Flow 20. Create a semantic metric, for example <code>Net Revenue = SUM(gross) - SUM(tax)</code> .	Path workspace_id; Body SemanticMetricRequest { name, formula, description?, related_tables? }	create_metric DatabaseManag metadata + ser
DELETE /api/workspaces/{workspace_id}/semantic/metrics/{metric_id}	Flow 20. Delete a semantic metric.	Path workspace_id, metric_id	delete_metric DatabaseManag metadata + ser
POST /api/workspaces/{workspace_id}/semantic/dimensions	Flow 20. Create a semantic dimension, for example <code>date = orders.created_at</code> .	Path workspace_id; Body SemanticDimensionRequest { name, table_name, column_name, description?, dim_type }	create_dimen: DatabaseManag metadata + ser
DELETE /api/workspaces/{workspace_id}/semantic/dimensions/{dimension_id}	Flow 20. Delete a semantic dimension.	Path workspace_id, dimension_id	delete_dimen: DatabaseManag metadata + ser
POST /api/workspaces/{workspace_id}/semantic/synonyms	Flow 20. Create a synonym mapping, for example <code>sales > revenue_usd</code> .	Path workspace_id; Body SemanticSynonymRequest { keyword, mapped_to, mapped_type }	create_synon: DatabaseManag metadata + ser
DELETE /api/workspaces/{workspace_id}/semantic/synonyms/{synonym_id}	Flow 20. Delete a synonym.	Path workspace_id, synonym_id	delete_synon: DatabaseManag metadata + ser
GET /api/workspaces/{workspace_id}/chat/history	Flow 21. Return the last 50 chat messages for this workspace.	Path workspace_id	get_chat_his app/core/age workspace met
DELETE /api/workspaces/{workspace_id}/chat/history	Flow 21. Clear all chat messages for this workspace.	Path workspace_id	clear_chat_h app/core/age workspace met
DELETE /api/workspaces/{workspace_id}/chat/history/{message_id}	Flow 21. Clear a specific chat message for this workspace.	Path workspace_id, message_id	delete_chat_l app/core/age workspace met
POST /api/workspaces/{workspace_id}/chat	Flow 21. Chat with workspace data through the RAG-to-SQL pipeline.	Path workspace_id; Body WorkspaceChatRequest { question, session_id?, title? }	workspace_ch app/core/age workspace met
POST /api/workspaces/{workspace_id}/report/execute_custom_sql	Flow 22. Execute raw SQL safely against the workspace schema and save history.	Path workspace_id; Body WorkspaceExecuteSqlRequest { title, sql_query }	execute_cust app/core/age workspace met
POST /api/workspaces/{workspace_id}/report/summarize	Flow 22. Generate an AI summary of a	Path workspace_id; Body WorkspaceReportSummarizeRequest	summarize_wor app/core/age

Endpoint	Flow / Trigger	Input	Handler / Coni
	workspace report.	{ question, sql_query, columns, data }	workspace met
POST /api/workspaces/{workspace_id}/report/share	Flow 22. Save a report snapshot and return a public share token.	Path <code>workspace_id</code> ; Body <code>_ShareReportRequest</code> { title, question, sql_query, columns, data, row_count, ai_summary }	<code>share_report_</code> <code>app/core/agei</code> workspace met
POST /api/workspaces/{workspace_id}/profile	Flow 19. Run auto-profiling on all tables in the workspace with LLM-generated descriptions and embeddings.	Path <code>workspace_id</code>	<code>profile_work:</code> <code>app/core/agei</code> semantic vector
POST /api/workspaces/{workspace_id}/profile/{table_name}	Flow 19. Run auto-profiling on a specific table.	Path <code>workspace_id</code> , <code>table_name</code>	<code>profile_table</code> <code>app/core/agei</code> semantic vector
GET /api/workspaces/{workspace_id}/relationships	Flow 20. List all relationships, including foreign-key and inferred relationships, for a workspace.	Path <code>workspace_id</code>	<code>list_workspa</code> > DatabaseMar metadata + ser
POST /api/workspaces/{workspace_id}/relationships	Flow 20. Create a user-defined relationship between two columns.	Path <code>workspace_id</code> ; Body <code>WorkspaceRelationshipRequest</code> { source_table, source_column, target_table, target_column, relationship_type }	<code>create_worksp</code> > DatabaseMar metadata + ser
DELETE /api/workspaces/{workspace_id}/relationships/{relationship_id}	Flow 20. Delete a relationship by ID.	Path <code>workspace_id</code> , <code>relationship_id</code>	<code>delete_worksp</code> > DatabaseMar metadata + ser
GET /api/workspaces/{workspace_id}/dashboard/stream	Flow 23. Stream dashboard widgets as NDJSON for progressive UI rendering. Each line is one widget JSON object. After all widgets are streamed, they are auto-persisted for instant cache on refresh. The final line is a summary event like { <code>"event"</code> : <code>"complete"</code> , <code>"widget_count"</code> : <code>N</code> }.	Path <code>workspace_id</code>	<code>stream_worksp</code> <code>app/core/agei</code> workspace met
GET /api/workspaces/{workspace_id}/dashboard	Flow 23. Get the workspace dashboard and auto-generate widgets on first load if empty.	Path <code>workspace_id</code> ; Query <code>regenerate</code>	<code>get_workspaci</code> <code>app/core/agei</code> workspace met
POST /api/workspaces/{workspace_id}/dashboard/save	Flow 23. Save the full workspace dashboard configuration including layout and widgets. This endpoint is intentionally non-destructive to widget data: when called from the frontend GridStack change handler, it only updates layout	Path <code>workspace_id</code> ; Body <code>WorkspaceDashboardSaveRequest</code> { name?, layout_json?, widgets? }	<code>save_workspai</code> <code>app/core/agei</code> workspace met

Endpoint	Flow / Trigger	Input	Handler / Coni
	positions and does not overwrite the stored config JSONB containing the full widget payload.		
POST /api/workspaces/{workspace_id}/dashboard/pin	Flow 24. Pin a chat-generated SQL result as a permanent dashboard widget.	Path workspace_id; Body WorkspacePinWidgetRequest { title, widget_type, chart_type?, sql_query, config?, origin_question? }	pin_widget_to app/core/agei workspace met
DELETE /api/workspaces/{workspace_id}/dashboard/widgets/{widget_id}	Flow 24. Remove a widget from the workspace dashboard.	Path workspace_id, widget_id	unpin_widget app/core/agei workspace met
POST /api/workspaces/{workspace_id}/dashboard/widgets/{widget_id}/refresh	Flow 24. Re-execute a widget's SQL query and return fresh data.	Path workspace_id, widget_id; Body req: Optional[RefreshRequest]	refresh_work: app/core/agei workspace met
POST /api/workspaces/{workspace_id}/dashboard/widget	Flow 24. Generate a single dashboard widget from a natural-language query.	Path workspace_id; Body WorkspaceWidgetRequest { query, widget_type_hint?, chart_type_hint? }	generate_worl app/core/agei workspace met
POST /api/workspaces/{workspace_id}/dashboard/widget/custom	Flow 24. Create a completely custom widget manually via SQL without LLM generation.	Path workspace_id; Body WorkspaceCustomWidgetRequest { title, sql_query, widget_type_hint, chart_type_hint? }	create_worksp -> app/core/agei workspace met
GET /api/workspaces/{workspace_id}/dashboard/schema	Flow 24. Return dimensions and measures for the Chart Builder sidebar.	Path workspace_id	get_workspac app/core/agei workspace met
POST /api/workspaces/{workspace_id}/dashboard/chart-builder	Flow 24. Generate a chart widget from explicit Chart Builder parameters.	Path workspace_id; Body WorkspaceChartBuilderRequest { chart_type, dimension, measure?, aggregation }	workspace_ch app/core/agei workspace met
GET /api/workspaces/{workspace_id}/etl-connection	Flow 12. Return the most recent connection_id used for this workspace so the UI can deep-link back into ETL editing.	Path workspace_id	get_workspac > DatabaseMar schema + regis

Core Backend Files To Know While Reading These Flows

- app/routers/files.py File upload, sheet extraction, file list, move, metadata, trash, chunks, cache delete.
- app/routers/query.py Legacy file query endpoint and batch execution.
- app/routers/dashboard.py Legacy dashboard generation, widget generation, filtering, compare mode, chart builder.
- app/routers/projects.py Project CRUD, dashboard access, active-file switching.
- app/routers/boards.py Boards, board-file uploads, board dashboards, publish/share flows.
- app/routers/etl.py External DB connection, preview, dry-run, full job execution, sync, pipeline editing.
- app/routers/workspace.py Workspace CRUD, catalog, semantic layer, relationships, chat, report, dashboard.
- app/routers/langfuse_share.py Public/shared routes and Langfuse support flows.
- app/core/ingestion.py File ingestion and metadata/embedding generation.
- app/core/routing.py Legacy intent routing and semantic cache.
- app/core/dashboard/kpi_builder.py Legacy dashboard and compare widget generation.
- app/core/agents/workspace_sql.py Workspace SQL reasoning, execution, report generation, dashboard widget generation.
- app/core/agents/profiler.py Workspace profiling and semantic enrichment.
- app/core/etl/* Source connectors, sandbox transform executor, and PostgreSQL loader.

Best Reading Order

If you want to understand the whole system with minimal confusion, read in this order:

1. this document
2. excel-ai-client/src/services/excelApi.ts

3. `excel-ai-client/src/services/workspaceApi.ts`
 4. `excel-ai-server-api/app/routers/files.py`
 5. `excel-ai-server-api/app/routers/query.py`
 6. `excel-ai-server-api/app/routers/dashboard.py`
 7. `excel-ai-server-api/app/routers/etl.py`
 8. `excel-ai-server-api/app/routers/workspace.py`
 9. `excel-ai-server-api/app/core/agent.py`
 10. `excel-ai-server-api/app/core/agents/workspace_sql.py`
 11. `excel-ai-client/src/pages/dashboard/SmartDashboard.vue`
 12. `excel-ai-client/src/pages/etl/index.vue`
 13. `excel-ai-client/src/pages/workspaces/dashboard/index.vue`
-

Final Mental Model

The simplest way to understand the project is to think in four primary journeys:

1. login -> upload file -> preview -> ask questions -> generate dashboard
2. organize files into projects/boards -> switch active file -> share/publish dashboard
3. connect source DB -> preview raw tables -> dry-run transform -> execute ETL -> sync updates
4. open workspace -> enrich semantics -> chat/report/dashboard over curated data -> share outputs

Once these journeys are clear, the codebase becomes much easier to navigate because every major file exists to support one or more of those flows.